

Úvod do UML

Karel Richta

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta, 2024

Softwarové inženýrství, SWI
03/2024, Lekce 5

<https://moodle.vspj.cz/course/view.php?id=202893>



Metodiky SI

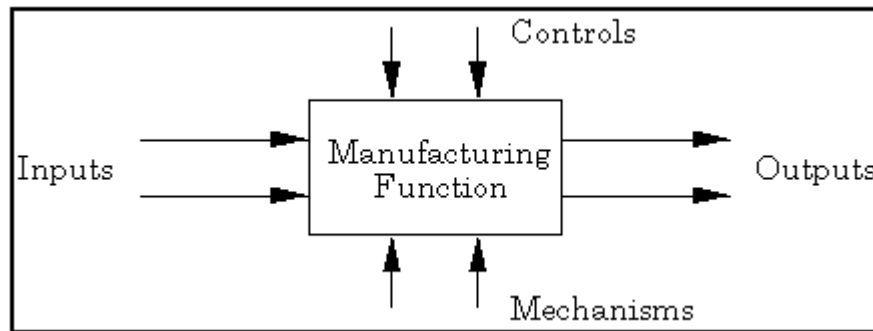
- **Metodika** = soubor metodických postupů a prezentačních technik (výrazových prostředků), které podporují systematický vývoj a údržbu programového díla, včetně plánování a řízení prací na programovém projektu.
- Čím se mohou metodiky lišit ?
 - **metodickými postupy** (procesy)
 - **prezentačními technikami** (notací)
 - **Nástroji**
- Začneme standardní notací: Unifikovaný modelovací jazyk UML ([ISO/IEC 19501:2005](#)).

Proč UML?

- UML je unifikovaný vyjadřovací prostředek pro dokumentaci obsahové části informatických projektů (dokumentace projektu obsahuje ještě projektovou dokumentaci, která popisuje projekt jako takový, zde UML žádné nástroje neposkytuje).
- UML umožňuje vyjádření dokumentace analýzy, návrhu i implementace.
- UML byl vyvinut na základě zkušeností s mnoha různými metodikami.
- UML byl určen zejména pro objektově-orientovaný přístup, ale umožňuje i vyjádření strukturovaného přístupu.

Alternativní notace

- Integrated Definition – IDEF (U.S. Air Force - <http://www.idef.com>)



Alternativní notace

- Architecture of Integrated Information Systems – ARIS
(prof. Scheer, SAP)
 - <http://www.ids-scheer.com>

Nejprve OO odhalili programátoři

- **Simula'67** (1962-67, poprvé zde byly uplatněny OO koncepty jako třída, objekt, dědění a dynamicky vázané metody)
- **Smalltalk** (1972-80, první jazyk založený zcela na konceptu komunikujících objektů, novinkou je též virtuální stroj jako interpret "byte code")
- **C++** (1983, 1998 - ISO, OO rozšíření C, novinkou je zpracování výjimek)
- **Eiffel** (1986, novinkou jsou kontroly invariantů, generické parametry, "garbage collection")

Pak se do toho vložili metodici

- 1.generace OO metodik:
 - Rumbaugh - OMT (diagramy tříd)
 - Coad/Yourdon - OOAD
 - Martin - OOIE
 - Booch - OOAD (diagramy komponent)
 - Wirfs/Brock - RDD (model odpovědností)
 - Jacobson - OOSE (model jednání - use cases)
- Každá metodika přinesla něco nového, byla v něčem lepší, ale zase něco postrádala $\Rightarrow$ snaha o kombinaci výhod

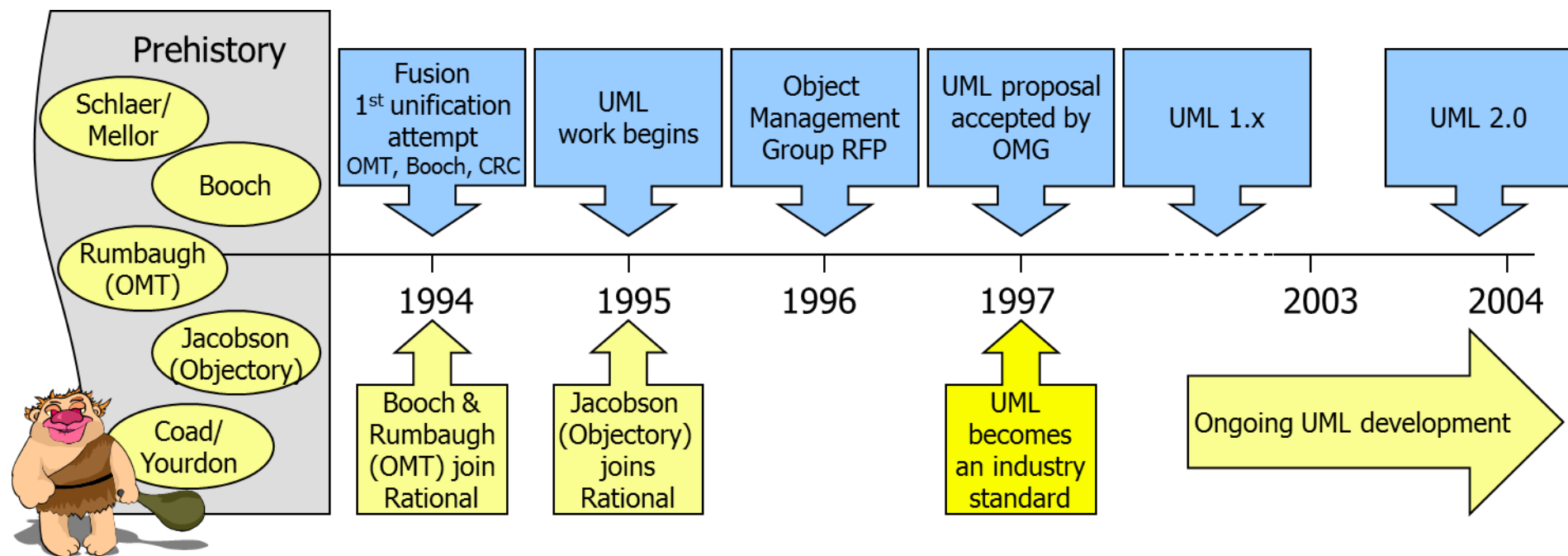
A zůstali u toho

- 2.generace OO metodik (kombinace výhod metodik 1.generace):
 - Coleman - Fusion (kompletní cyklus)
 - Rumbaugh/Booch - UM (Unified Method 0.8)
- Zůstaly rozdíly v prezentačních technikách, každá metodika používá svou notaci $\Rightarrow$ snaha o unifikaci
- 3.generace (unifikace prezentačních technik):
 - Rumbaugh/Booch - UM (Unified Method 0.8)
 - 1996 - Rumbaugh/Booch/Jacobson - UML Unified Modeling Language 1.0)
 - 1997 - OMG standard UML 1.1

Historie UML

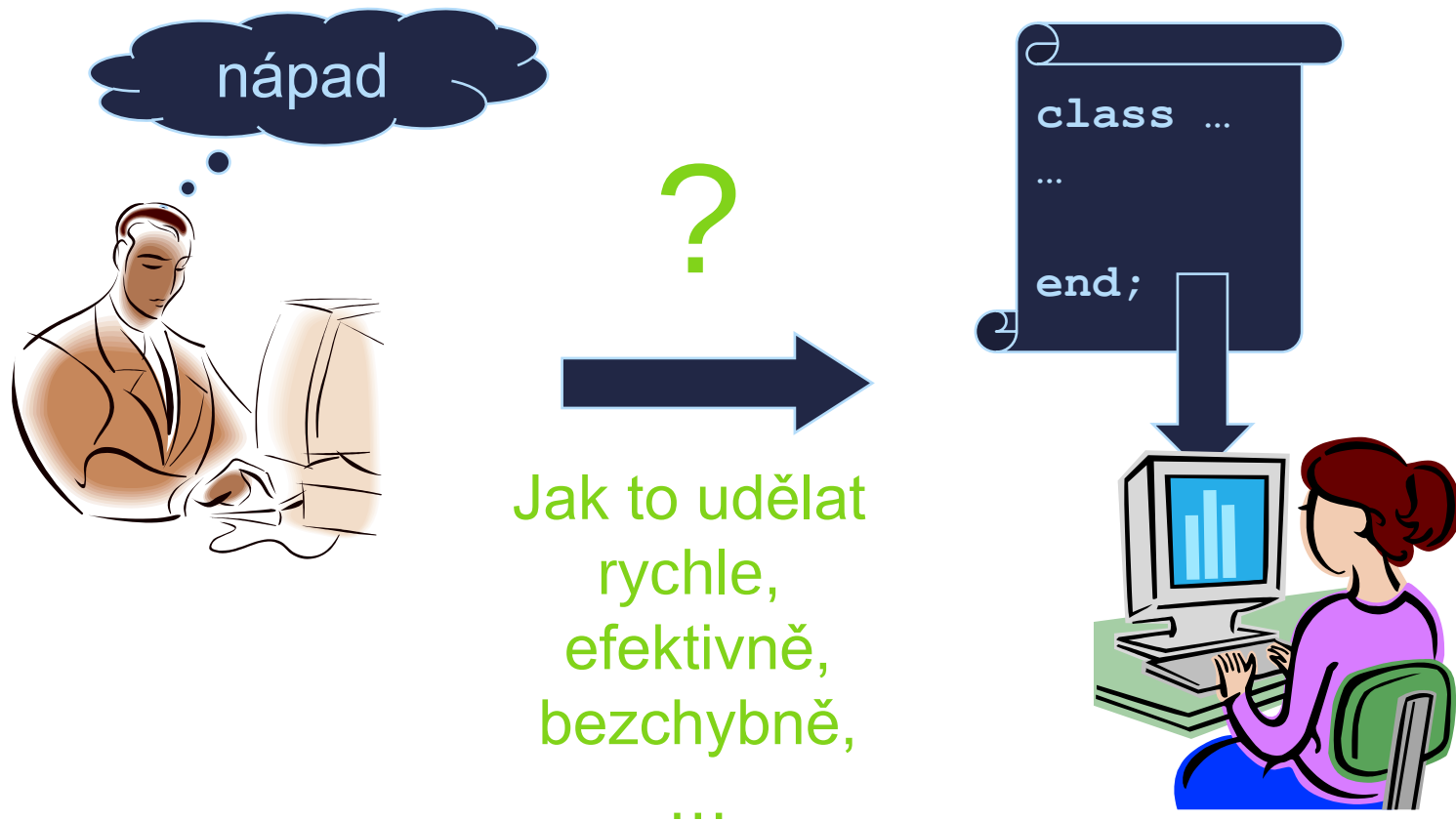
- **1994** – zahájení vývoje UML (Rumbaugh, Booch)
- **1995** – notace pro Unified Method 0.8, připojuje se Jacobson a model jednání, Rational Unified Process
- **1996** – UML 1.0 (zabudování připomínek, kooperace)
- **1997 – UML 1.1 (preciznější sémantika, přijat jako standard OMG)**
- **1999** – UML 1.3 (upřesněná verze 1.1)
- **2001** – UML 1.4 (komponenty)
- **2003 – UML 1.5 (diagramy aktivit, sémantika akcí)**
- **2005 – UML 2.0 (nové diagramy, změny)** - původně plánované ukončení květen 2002, ale 😊 (příliš mnoho hlav, obtížné)
- **2015 – UML 2.5** (další snaha o upřesnění sémantiky)

Historie UML graficky



Zdroj: © Arlow, Neustadt: UML 2 a objektově orientovaný návrh.

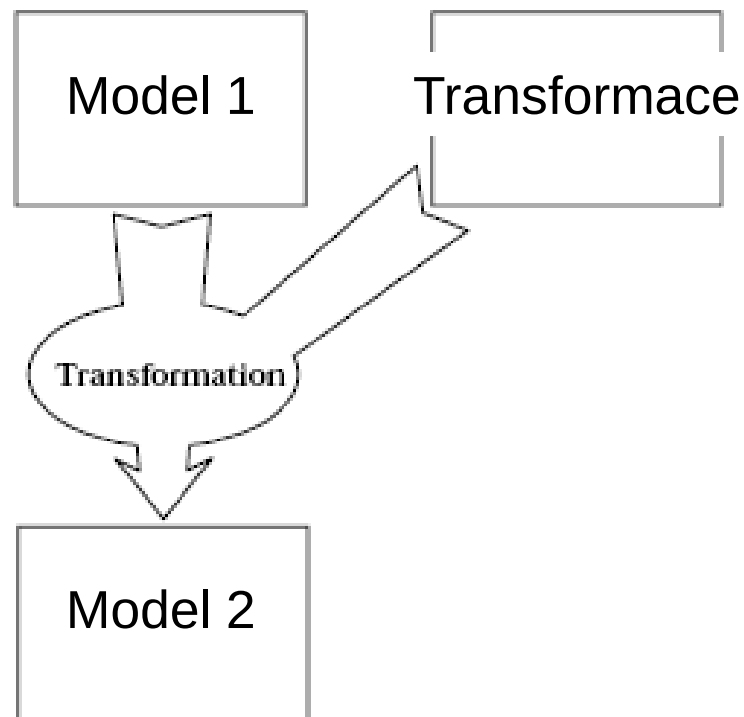
Hlavní problém tvorby software



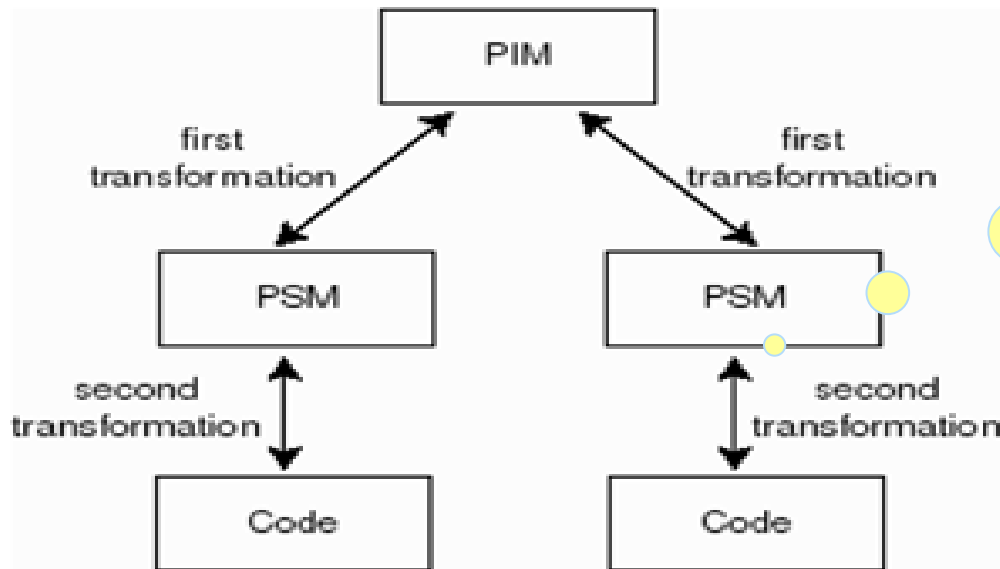
Jak postupovat?

- Začít hned kódovat?
- Nejdříve modelovat, pak kódovat?
- Zdá se, že vývoj software přešel od kódem řízeného stylu ke specifikací řízenému stylu.
- Nejnovější trend konsorcia OMG (Object Management Group) je modelem řízený vývoj (MDD – Model Driven Development), nebo také modelem řízená architektura (MDA), příp. modelem řízené inženýrství (MDE).

Vývoj software jako posloupnost transformací

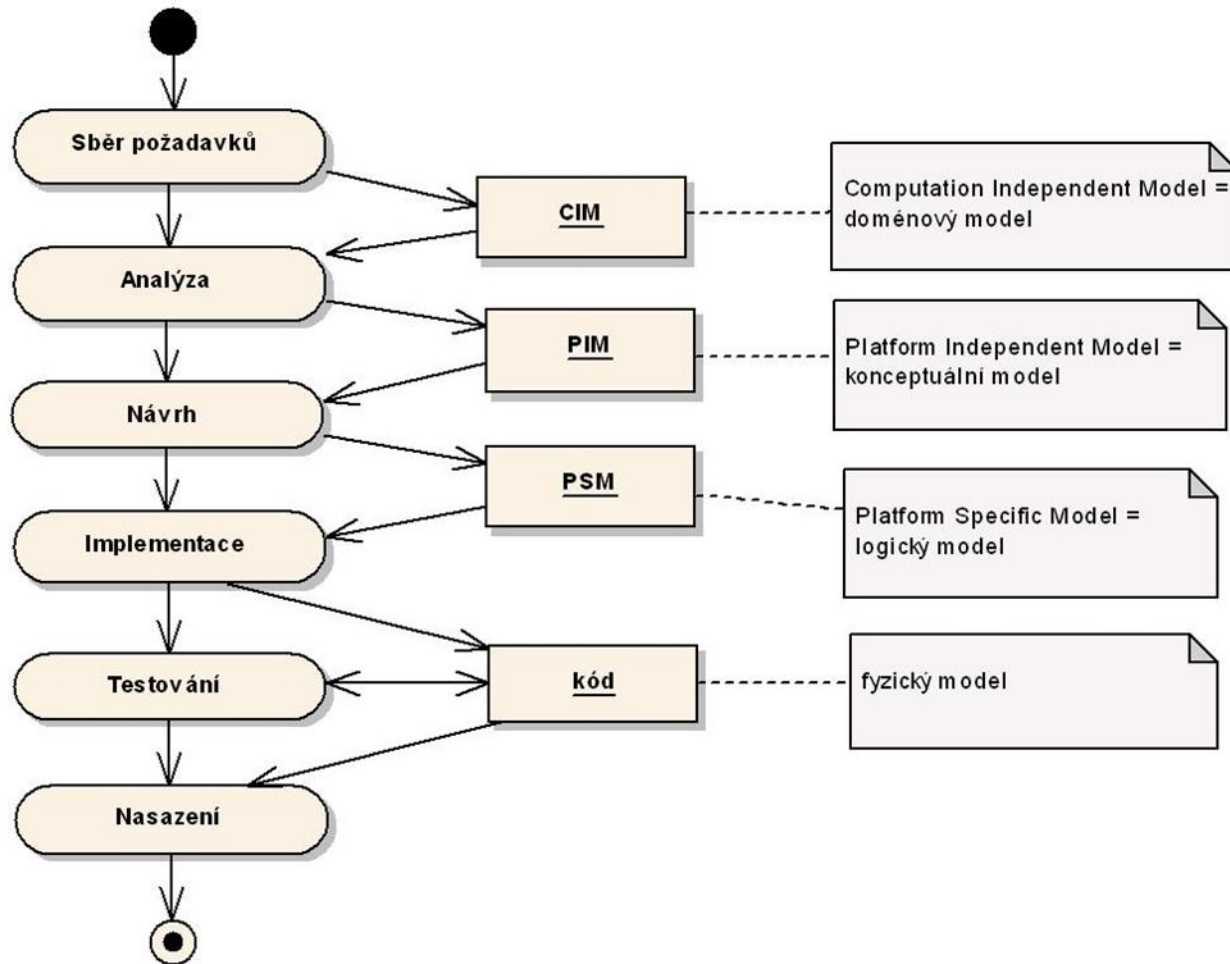


Modelem řízený vývoj (Model Driven Development)



Pracuje se
co nejvíce
s modelem

Modelem řízený vývoj v UML



6 dobrých zásad tvorby SW

- Vyvíjej iterativně a přírůstkově
- Řádně rozpozněj a zpracuj požadavky
- **Modeluj vizuálně (=> UML)**
- Používej komponentové technologie
- Kontinuálně ověřuj kvalitu, testuj
- Snaž se být připraven na změny

Best practices
IBM Rational

Co to je UML?

- **UML (Unified Modeling Language) je:**

„Standard konsorcia OMG (Object Management Group) pro záznam, vizualizaci a dokumentaci artefaktů systémů s převážně softwarovou charakteristikou“.

Co UML není?

- UML není žádná metodika – slouží pouze pro vyjádření artefaktů určitého typu, které metodiky požadují. Jsou metodiky, které UML intensivně využívají (např. UP, RUP), jiné nikoliv.
- UML není všespasitelné (U znamená unifikovaný, nikoli univerzální).

Tři úrovně využívání UML

- Jako **notaci** pro zachycení artefaktů, na kterých se je potřeba domluvit. Lze jej využít jak přímo (nakreslíme obrázek, abychom si upřesnili implementaci), tak zpětně (obrázek nám poslouží k pochopení existujícího kódu).
- Jako **dokumentační prostředek**, ve kterém dokumentujeme systém, či jeho části.
- Jako **programovací jazyk**, ze kterého se skutečná implementace (přesněji asi jen její kostra) generuje.

Složky UML

- **Elementární prvky** (infrastruktura)
- **Diagramy UML** (superstruktura)
- **OCL** (Object Constraint Language – jazyk pro popis omezení)
- **Výměnný formát** (buď jen model – XMI, IDL, nebo včetně diagramů konvertovaných do SVG)

Lexikální elementy UML

Čtyři základní druhy elementů:

řetězec - posloupnost znaků, znaková sada není omezena, v některých případech doporučeno ASCII

ikona - grafický symbol zastupující element, neobsahující žádné složky

2D-symbol - grafický symbol zastupující element, který má obsah, případně rozdělený na oblasti

spojnice - path - posloupnost úseček, které navazují, mohou mít zvýrazněny koncové body a mohou incidovat s hranicí 2D-symbolů - slouží k vyznačení propojení

Identifikace elementů UML

elementy potřebujeme označovat - potřebujeme jména objektů, tříd, atributů, metod, ...

jméno - identifikátor (včetně '_' nebo '-')

josefNovak, josef_novak, josef-novak

kvalifikované jméno - jméno se vztahuje k nějakému kontextu (scope), explicitně:

Ucetnictvi::Objednavka

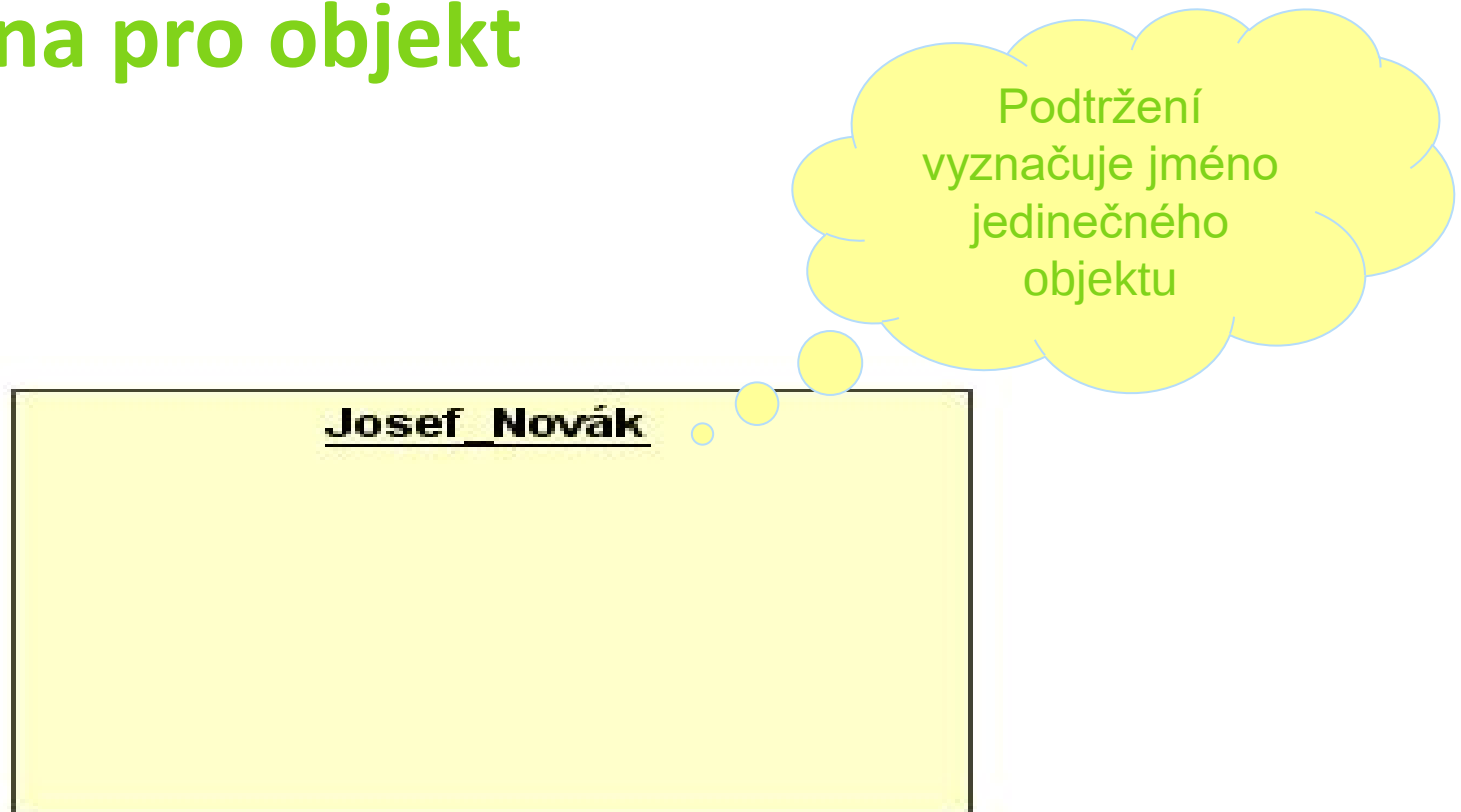
návěští - řetězec přidružený ke grafickému symbolu

klíčové slovo

<<keyword>> (guillemets)

Nejprve některé ikony

Ikona pro objekt



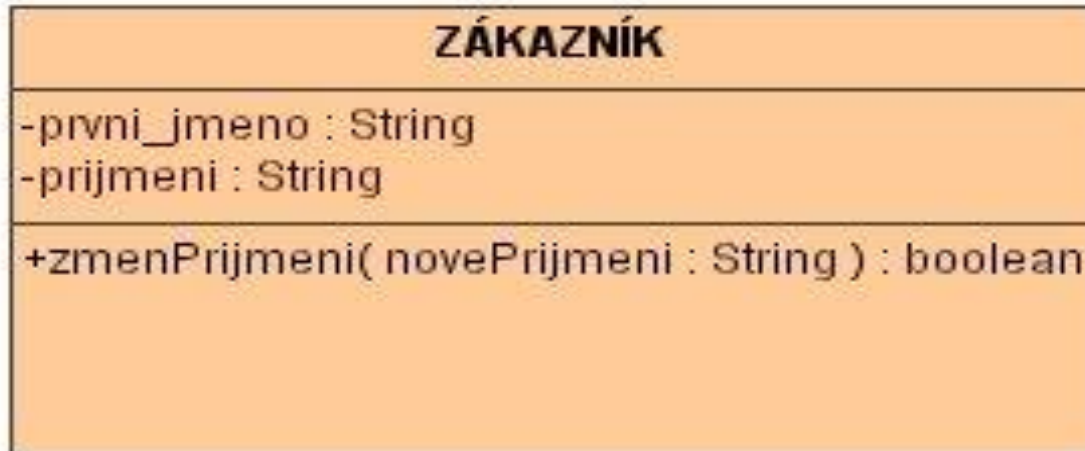
Jedinečný objekt identifikovaný svým jménem

Ikona pro třídu



Třída zahrnuje objekty stejného typu – se stejnými atributy a operacemi

2D-symbol pro třídu



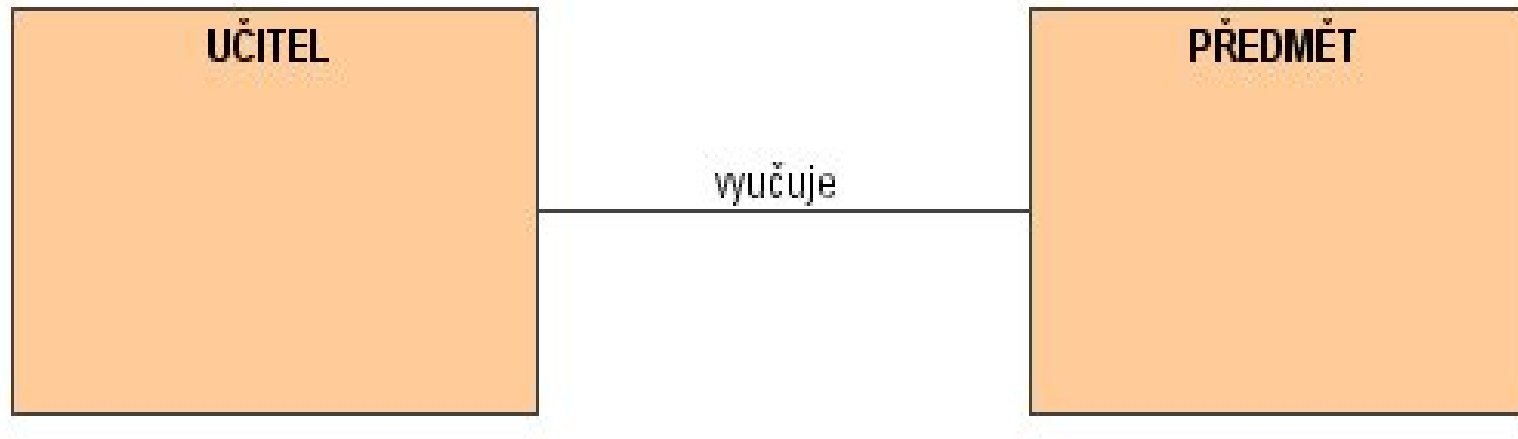
Třída s atributy a operacemi

Ikona pro balík (package)



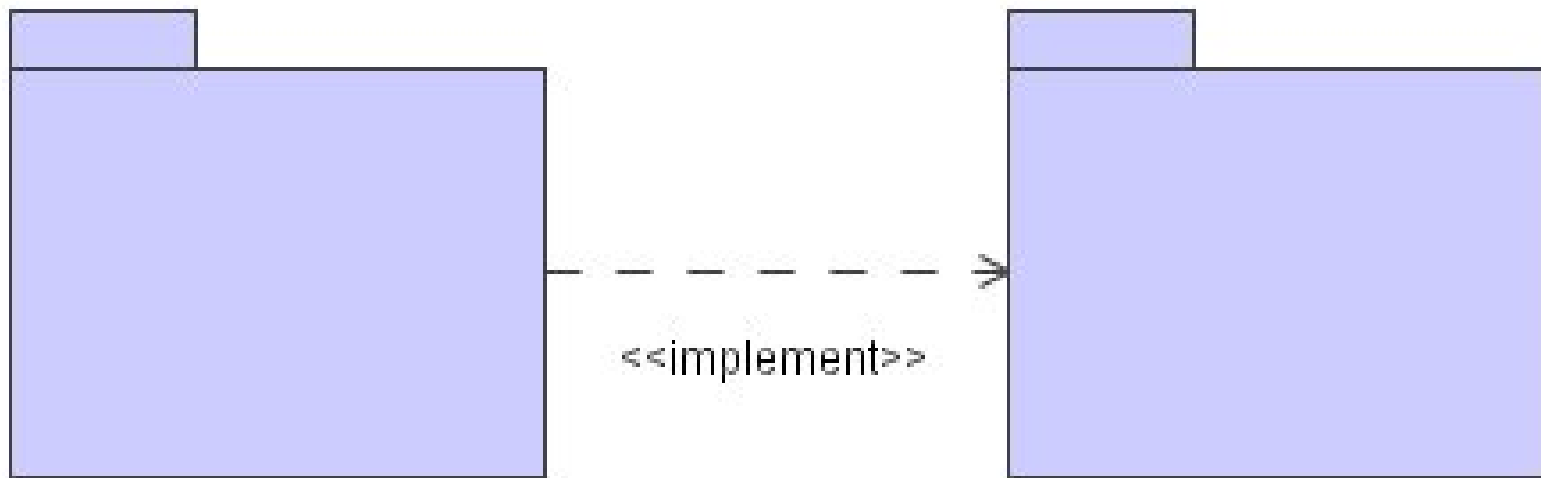
Logická skupina elementů

Spojnice



Vyjadřuje vztah (v tomto případě symetrický vztah mezi třídami)

Závislost



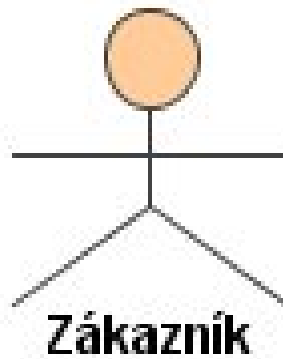
Vyjadřuje závislost (v tomto případě, že něco implementuje něco jiného)

Ikona pro případ užití (use case)



Záznam o případě použití – vně systému může nastat událost, na kterou musí systém reagovat

Ikona pro aktéra



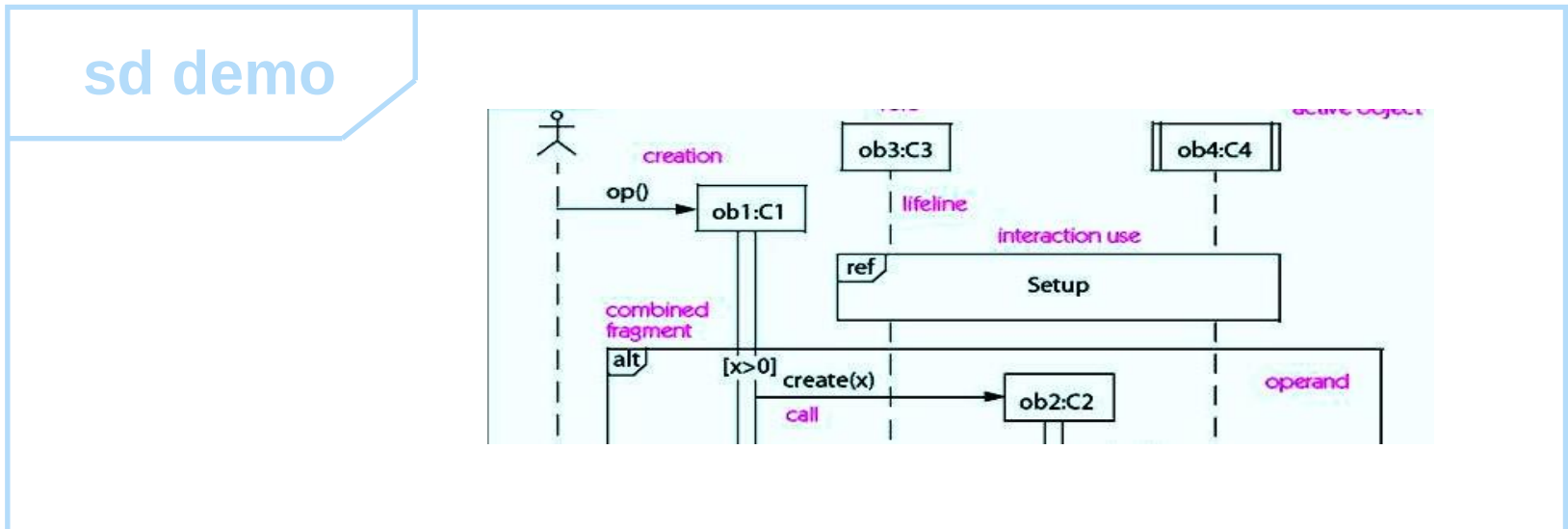
Uživatelská role nebo spolupracující systém

Rámec (frame)



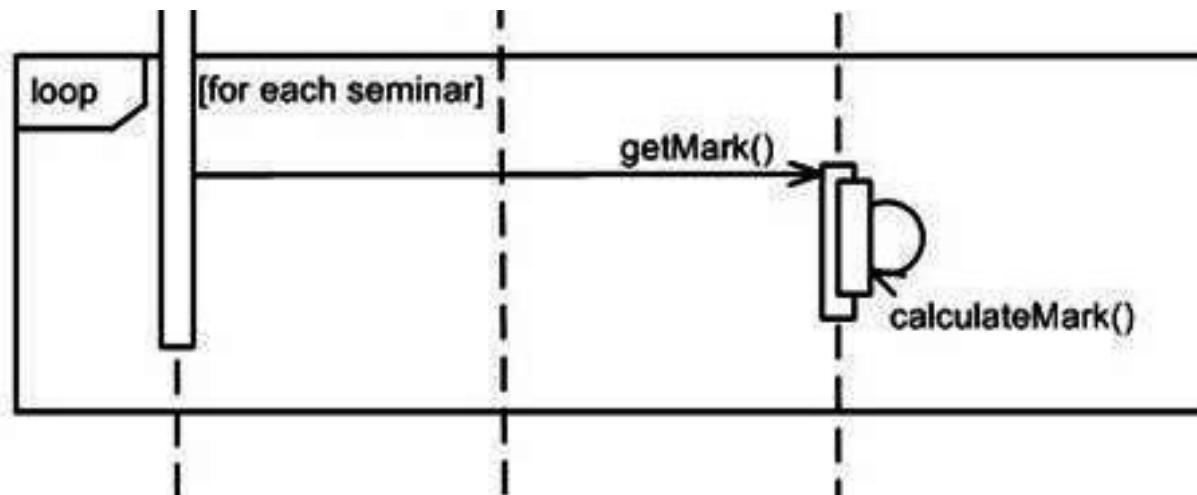
Dá se použít pro celý diagram, nebo pro nějaký jeho prvek (např. zde pro balík P), nebo např. jako interakční rámec ve scénáři. Návěští popisuje typ rámce (zde Package), identifikátor rámce (zde P) a případné parametry (zde nic).

Příklad použití rámce pro diagram



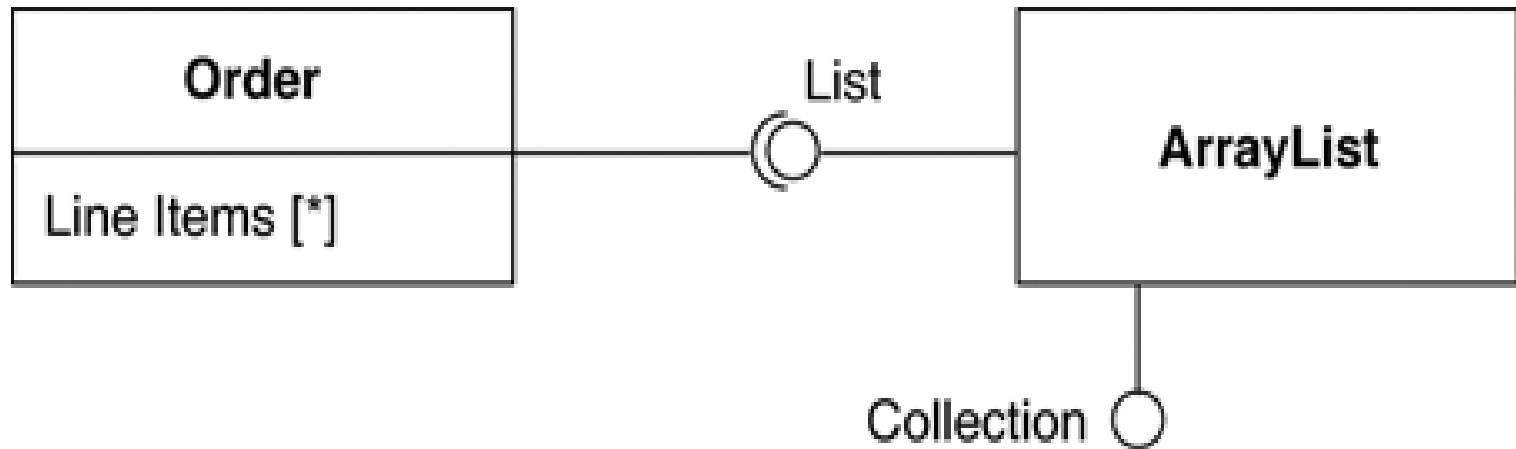
Zde je rámcem označen celý diagram typu „sd“ = „sequence diagram“ = scénář, který se jmenuje „demo“.

Příklad použití rámce ve scénáři



Rámec je zde typu „loop“ (smyčka) a nemá jméno. Vyjadřuje, že se určitá část scénáře opakuje pro každý seminář.

Interface



Nová ikona pro rozhraní (interface), vyjadřující přesněji, kdo je poskytovatel (ball) a kdo konzument (socket).

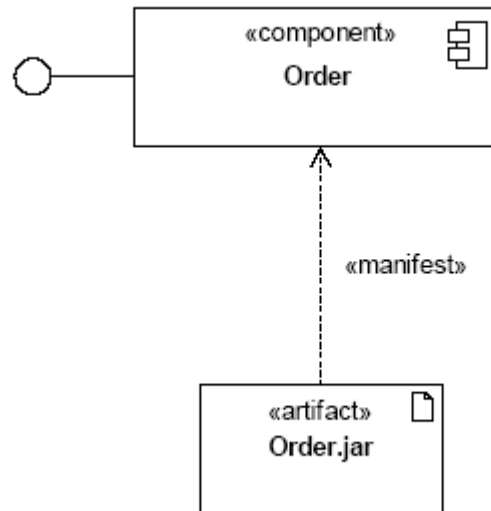
Artefakt

- Artefakt je specifikace fyzického kousku informace, která je použita nebo produkována při vývoji softwaru nebo při jeho nasazení a použití.
- Příkladem artefaktu může být modelový soubor, zdrojový soubor, skript, vykonatelný program, tabulka v databázi, libovolná součást dodávky, textový dokument, či zpráva.
- Instance artefaktu:



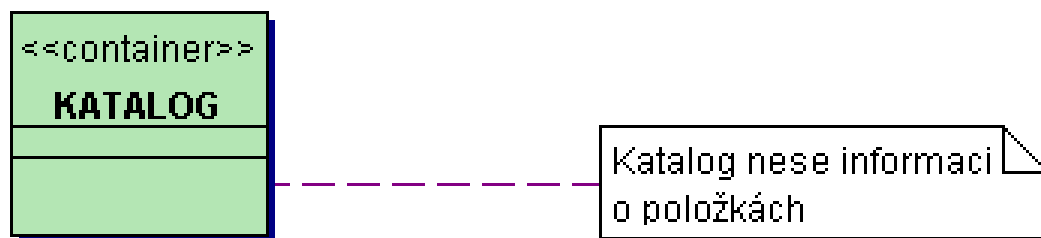
Použití artefaktu v diagramech

- Komponenta „Objednávka“ je „manifestována“ artefaktem „Objednávka.jar“



Poznámky

- Poznámka obsahuje libovolný text
- Zobrazuje se jako obdélník s „přehnutým rohem“
- Může být přidružena k elementu
- Může obsahovat stereotyp (např. **<<constraint>>** - pak se jedná o specifikaci omezení)



Výrazy v UML

- Výrazy v UML jsou řetězce sestavené podle jistých pravidel (syntaxe výrazů není striktní, je doporučeno používat OCL)
- Výraz představuje formuli, kterou musí být možno v daném jazyce vyhodnotit (má sémantiku)
- Definice UML zahrnuje definici OCL (Object Constraint Language), který je použit pro definici sémantiky UML (metamodel UML)

Navigace v OCL

objekt.selektor

- označuje položku daného objektu (atribut, jméno role)

objekt.selektor [kvalifikátor]

- označuje objekt vybraný podle kvalifikátoru (zobecněné pole)

sada -> select (logická podmínka)

- označuje objekty vybrané z dané sady podle logické podmínky:

Firma.zamestnanci ->

select(funkce="ucetni")

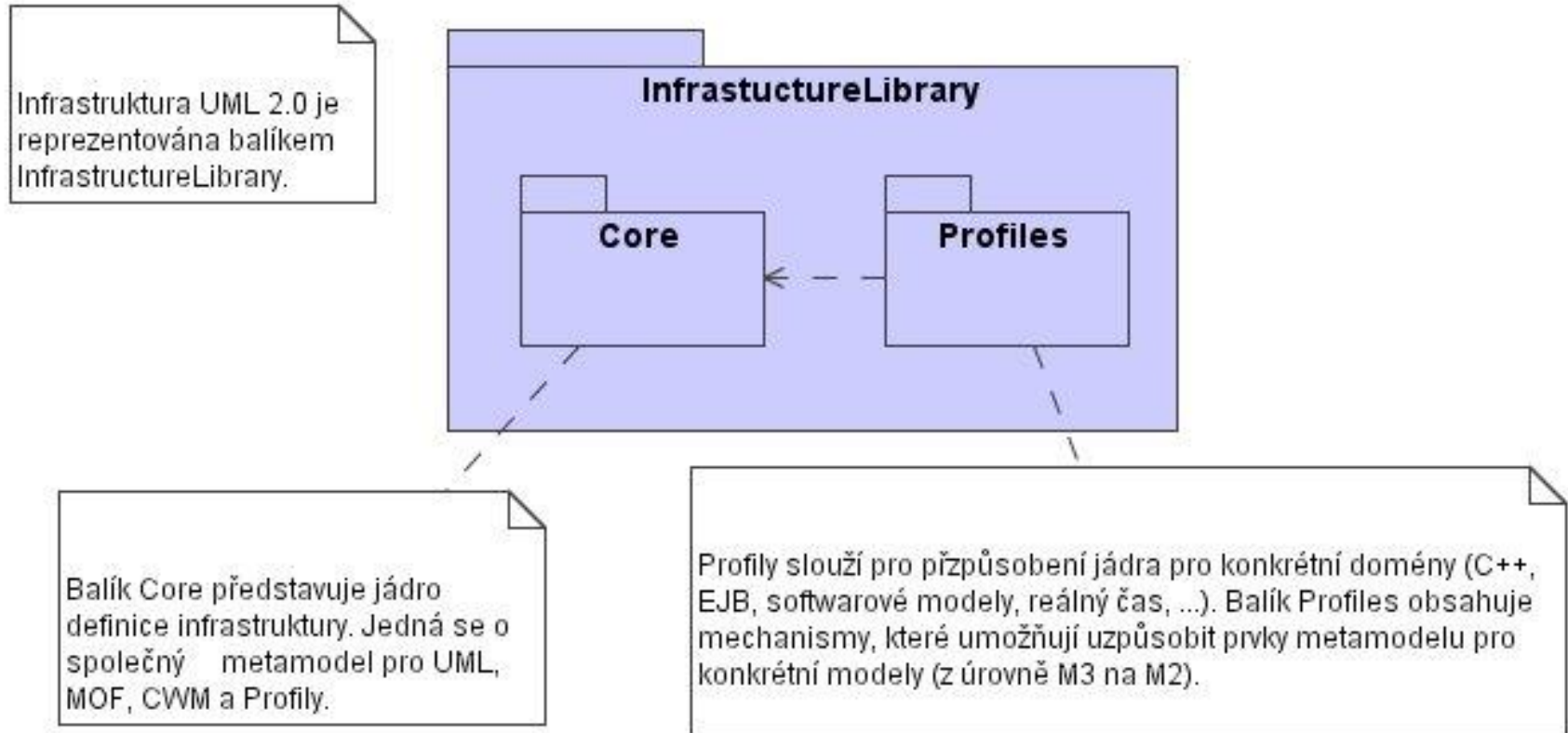
Struktura UML 2

Specifikace standardu UML 2 je rozdělena do 4 částí:

- **define infrastruktury** – notace UML (syntax), definuje základní elementy, jádro UML společné pro UML a související standardy, např. MOF či CWM
- **define superstruktury** - sémantika definovaná pomocí metamodelu, popis prvků metamodelu, konstrukty používané uživateli UML – elementy diagramů a diagramy
- **define výměnné struktury** (diagram interchange) – pro export a import modelů a diagramů, formát pro výměnu dokumentů (diagramů) mezi různými nástroji, specifikace převodu do výměnných formátů (CORBA IDL, XML DTD)
- **Object Constraint Language** (OCL) – jazyk pro formálně přesný popis různých omezení platných v modelu

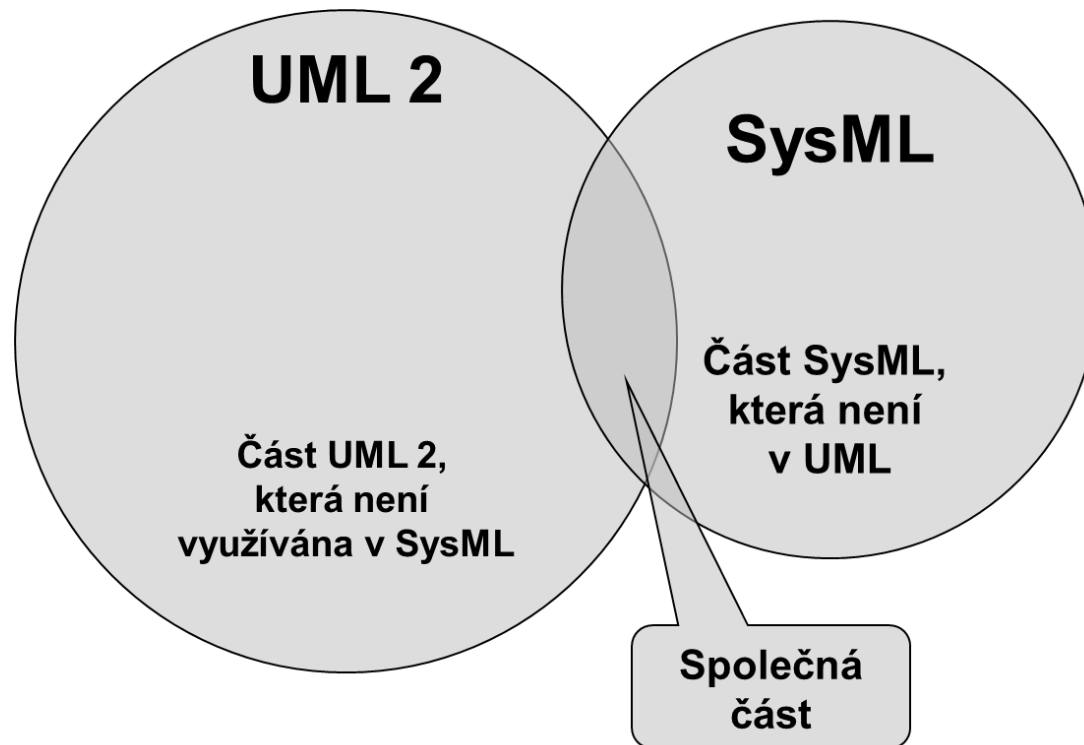
<http://www.uml.org>

Infrastruktura UML (2.0)

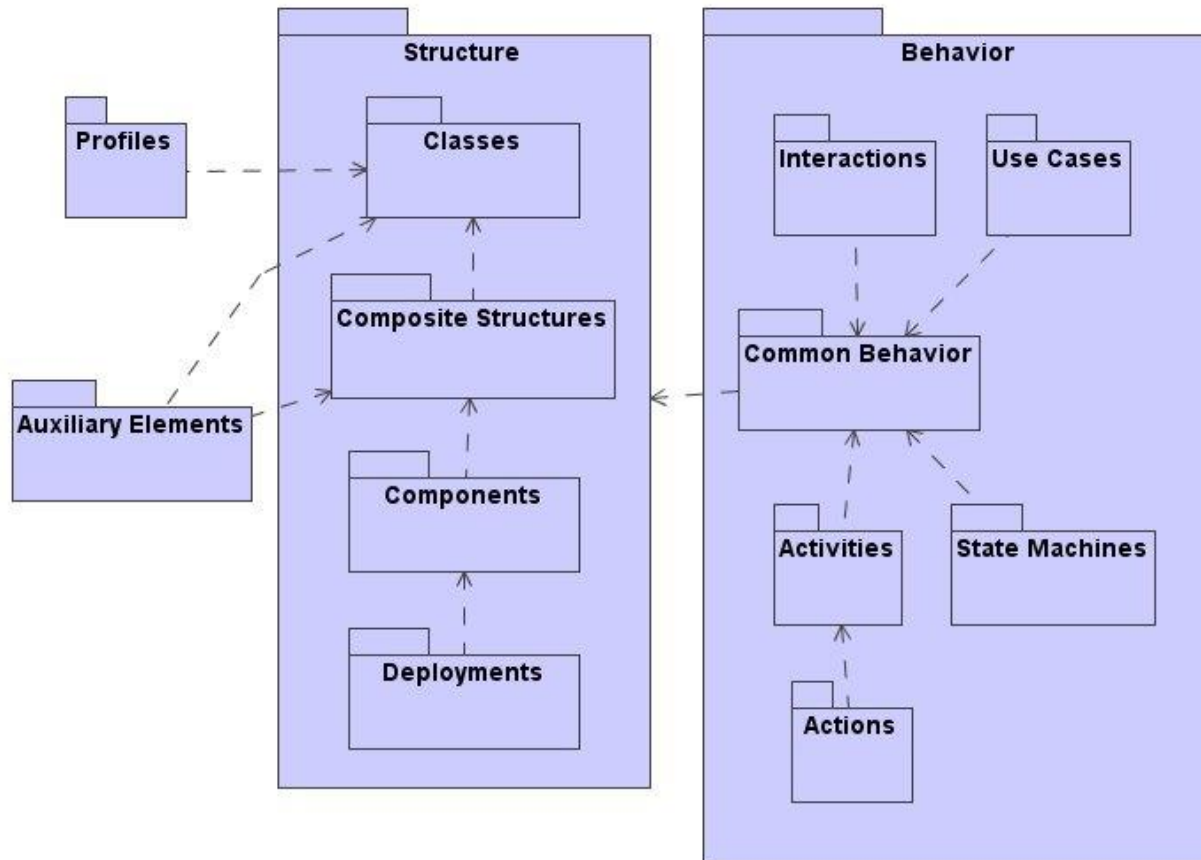


Profily v UML

Příklad: SysML je rozšíření UML, které využívá jen některé diagramy UML a přidává nové diagramy, které v UML nejsou.



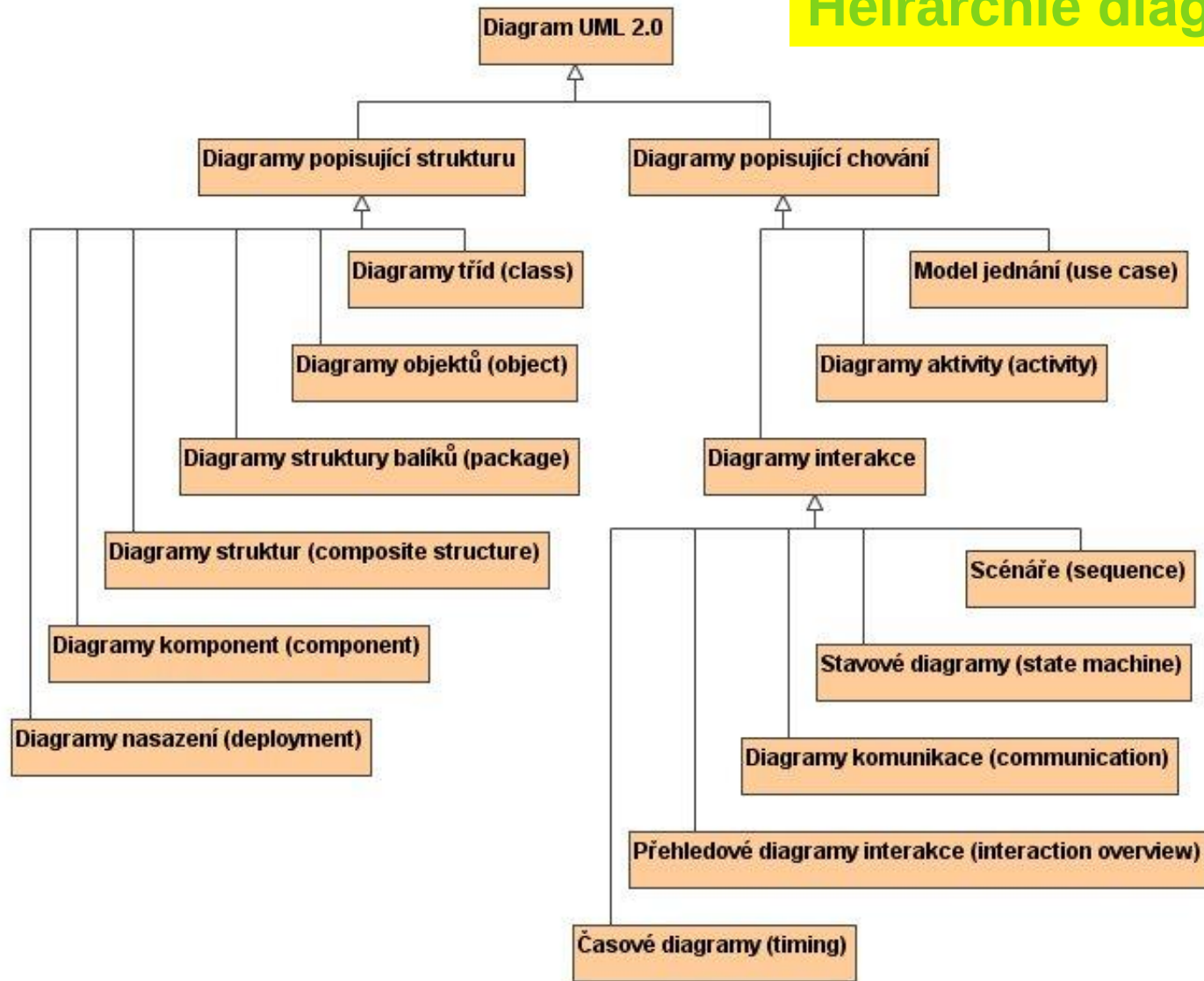
Superstruktura UML (2.0)



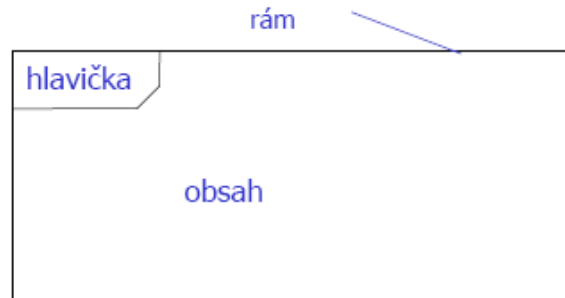
Diagramy verze 1.5 a 2.0

UML 1.5	UML 2.0
Diagram tříd (class diagram)	Diagram tříd (class diagram)
Diagram objektů (object diagram)	Diagram objektů (object diagram)
Model jednání (use case diagram)	Model jednání (use case diagram)
Scénář (sequence diagram)	Scénář (sequence diagram)
Diagram kolaborace (collaboration diagram)	Diagram komunikace (communication diagram)
Diagram aktivity (activity diagram)	Diagram aktivity (activity diagram)
Stavový diagram (statecharts diagram)	Stavový diagram (state machine diagram)
Diagram komponent (component diagram)	Diagram komponent (component diagram)
Diagram nasazení (deployment diagram)	Diagram nasazení (deployment diagram)
	Diagram struktury (composite structure diagram)
	Diagram struktury balíků (package diagram)
	Přehledový diagram interakce (interaction overview diag.)
	Časový diagram (timing diagram)

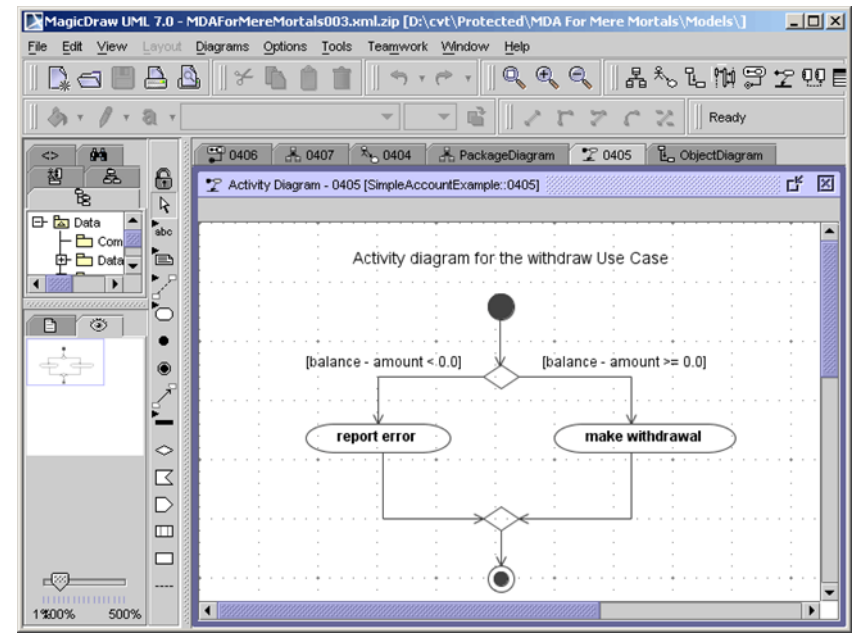
Heirarchie diagramů



Syntaxe diagramu – rám (frame)



syntaxe hlavičky: <druh> <jméno> <parametry>
<druh> a <parametry> jsou volitelné

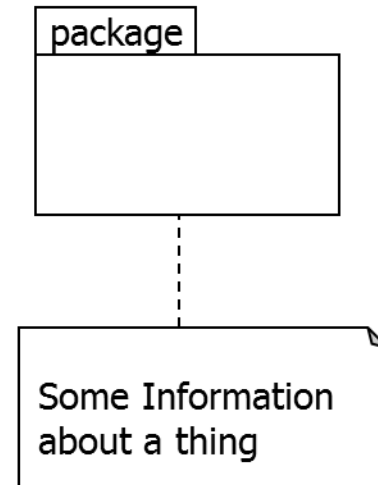


Stavební bloky UML

- Prvky
 - Modelové elementy (ikony, 2D-prvky)
- Vztahy
 - Vazby mezi elementy (spojnice)
- Diagramy
 - Pohledy zobrazující určitou stránku modelu
 - Model je za všemi pohledy

Prvky

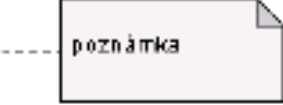
- Strukturální prvky – podstatná jména modelu UML
 - Třídy, rozhraní, kolaborace, případy užití, komponenty, uzly
- Prvky chování – slovesa modelu UML
 - Interakce, stavové automaty
- Seskupující prvky
 - Balíky (Packages)
 - Modely, frameworky, subsystémy
- Anotační prvky
 - Poznámky
 - Příznaky (Tags)
 - Stereotypy
 - Omezení



Elementy a vztahy

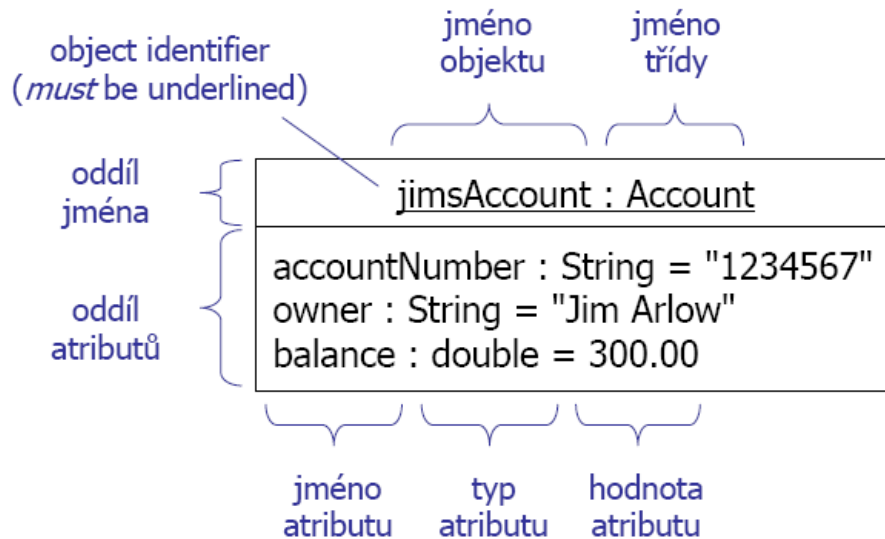
- Každý model je dokumentován sadou pohledů.
- Model v UML je dokumentován sadou diagramů, které dokumentují určité rysy modelu.
- Každý diagram je sestaven z jistých elementů a vztahů mezi nimi.
- Obecně se v UML připouští, aby v každém modelu byl použit libovolný element.
- Ne všechny kombinace jsou ale smysluplné, vždy určitá kombinace elementů a vztahů představuje superstrukturu diagramu jistého typu. Některé elementy a vztahy jsou použitelné obecně.

Základní obecné elementy

Element (prvek)	UML syntaxe	UML sémantika
poznámka		doplňující podrobnější popis elementu nebo diagramu
Balík (package)		logická skupina elementů
objekt (instance)		konkrétní objekt
Třída		třída objektů, které mají něco společného

Tab. 1: Obecné elementy UML

Základní element - objekt



varianty

(neuvevedeme oddíl atributů)

jméno objektu
a třídy

jimsAccount : Account

pouze jméno
objektu

jimsAccount

pouze
jméno třídy

: Account

anonymní objekt

Základní vztahy

Vztah (relationship)	UML syntaxe	UML sémantika
vztah (asociace)		vztah mezi elementy (symetrický)
souvislost		související elementy (připojení poznámky)
závislost		jeden element závisí na jiném elementu
agregace		Element obsahuje jiný element (celek-část)
kompozice		silnější agregace (stejný životní cyklus)
obsahuje		zdrojový element obsahuje cílový element
generalizace		zdrojový element je specializací cílového
realizace		zdrojový element je realizací cílového

Tab. 2: Obecné vztahy v UML

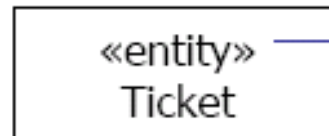
Ornamenty elementů

Ornamenty (adornments):

- stereotypy
- příznaky (tags)
- omezení (constraints)

Stereotypy

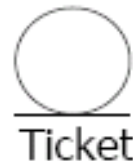
jméno sterotypu v
<< >>



stereotype

preferováno

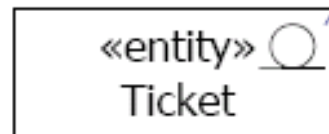
speciální ikona



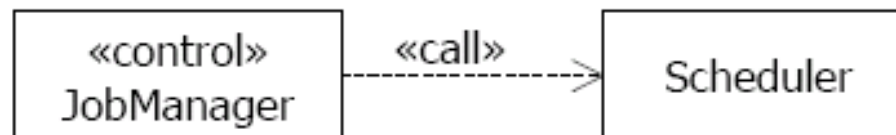
icon

preferováno

jméno i ikona

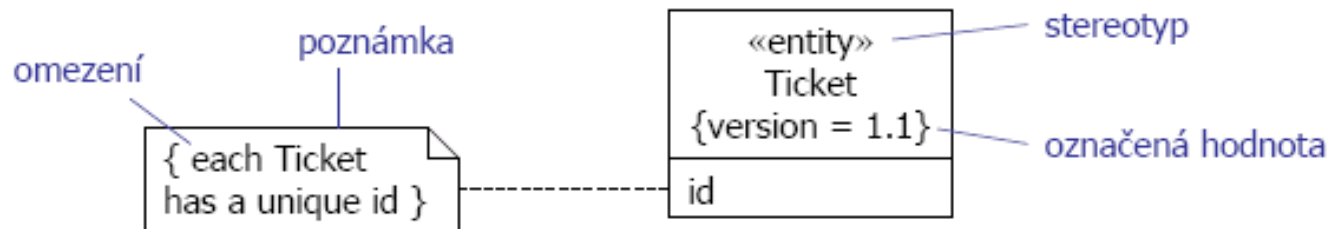
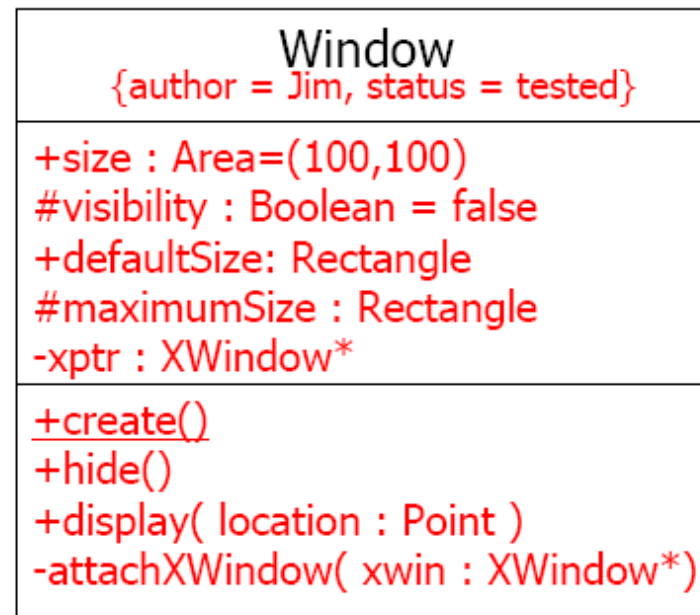


stereotypovaný
vztah



Příznaky, omezení

Window



Modelový příklad – Hotelový rezervační systém (HRS)

- Uvažme jako ilustrační příklad projekt hotelového rezervačního systému, na kterém budeme ukazovat možnosti jednotlivých diagramů.
- Vize projektu HRS říká, že zákazníkem je společnost provozující řetězec ubytovacích zařízení.
- Vzhledem k tomu, že společnost úspěšně expanduje a rozšiřuje svojí působnost, chtěla by nahradit stávající evidenci rezervací, která není vyhovující a způsobuje velké finanční náklady, evidencí novou.
- Nový systém je určen pro řízení rezervací ubytování, webový přístup pro prohlížení ubytovacích zařízení a pokojů, sledování aktuálního stavu rezervací a vytváření nových rezervací.

Modelový příklad – HRS

- Dle zadání by předmětem zakázky měla být dodávka tří komponent:
 - **aplikace pro recepci v hotelech (HotelApp),**
 - **webové aplikace pro přístup zákazníků ze sítě (WebApp) a**
 - **kiosk, sloužící pro zveřejňování lokálních informací – restaurace, kulturní akce apod. (KioskApp).**
- Dle popisu v dokumentaci není komponenta KioskApp závislá na ostatních aplikacích a disponuje vlastními procesy, které mohou být dosti rozsáhlé. Detailní specifikace může znamenat prodloužení nasazení očekávané aplikace, což pro zákazníka přináší další finanční ztráty.
- Rezervace hotelových pokojů a práce s ubytovacími objekty je pro daný typ zákazníka klíčová a implementací omezené části požadovaného systému se můžeme vyhnout případným ztrátám, které mohou způsobit rizika uvedená v dokumentaci.

Čím začneme?

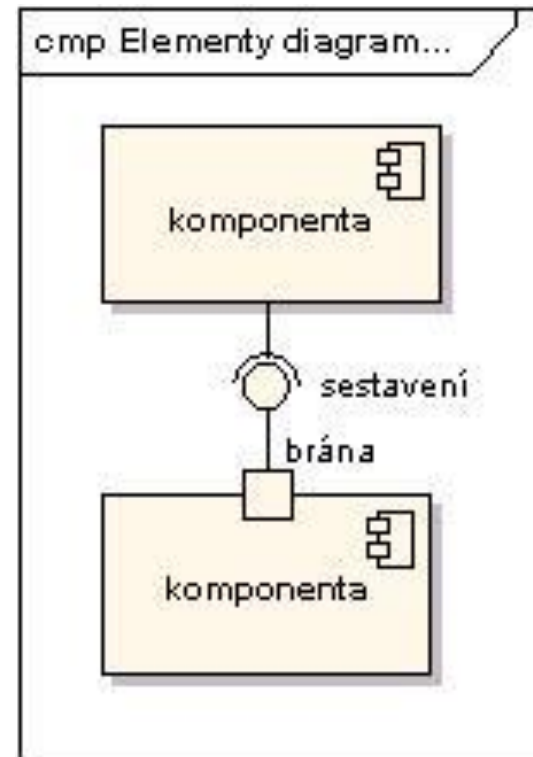
- Ve funkčním přístupu se obvykle začíná diagramem případů užití, scénáři pro použití (**modelem jednání**).
- V datově-orientovaném přístupu se obvykle začíná diagramem tříd (**modelem dat**).
- My začneme diagramem komponent a diagramem nasazení (**modelem architektury**).

Diagramy komponent

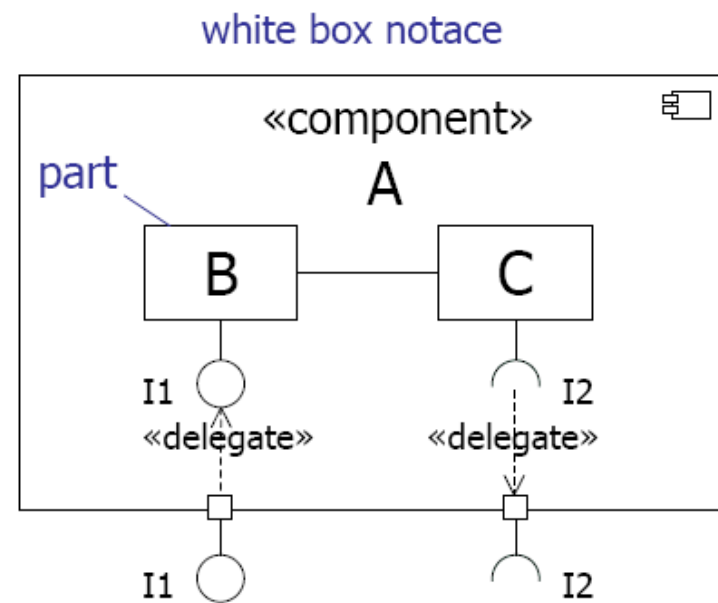
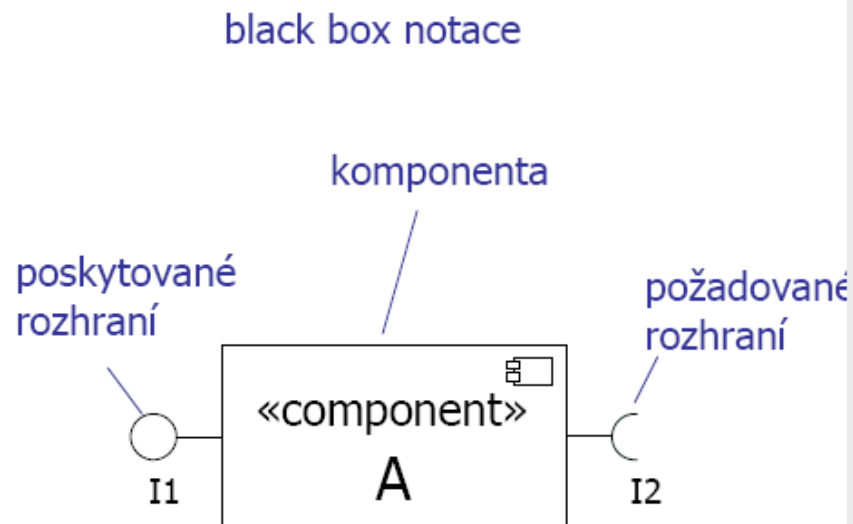
- Co všichni známe:

Komponenta je modulární část systému zapouzdřující svůj obsah. Její projev je nahraditelný – je to „černá skříňka“ projevující se pomocí požadovaných a poskytovaných rozhraní.

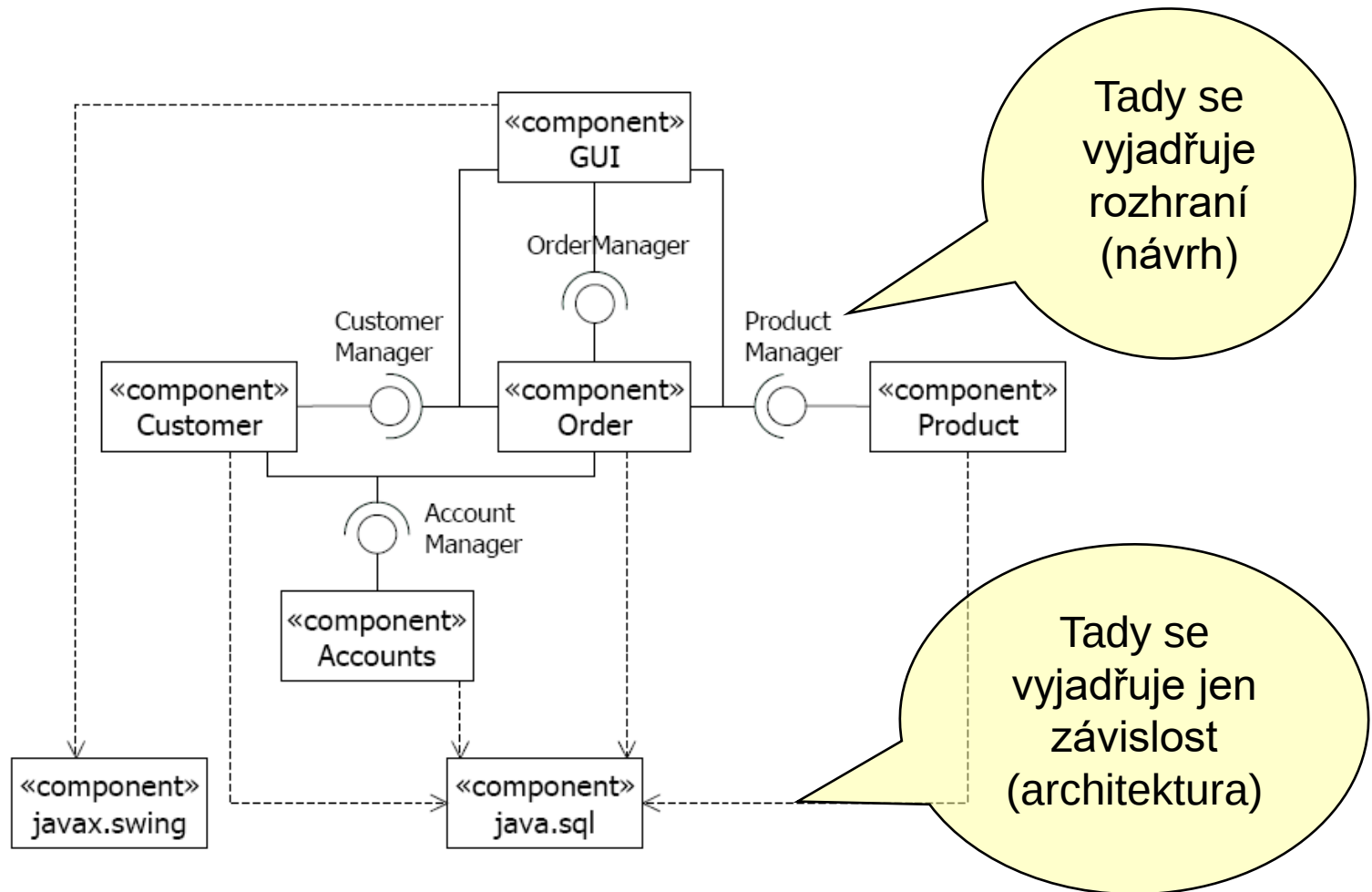
- Co není tak obvyklé?
- Co není známo vůbec?



Notace komponent

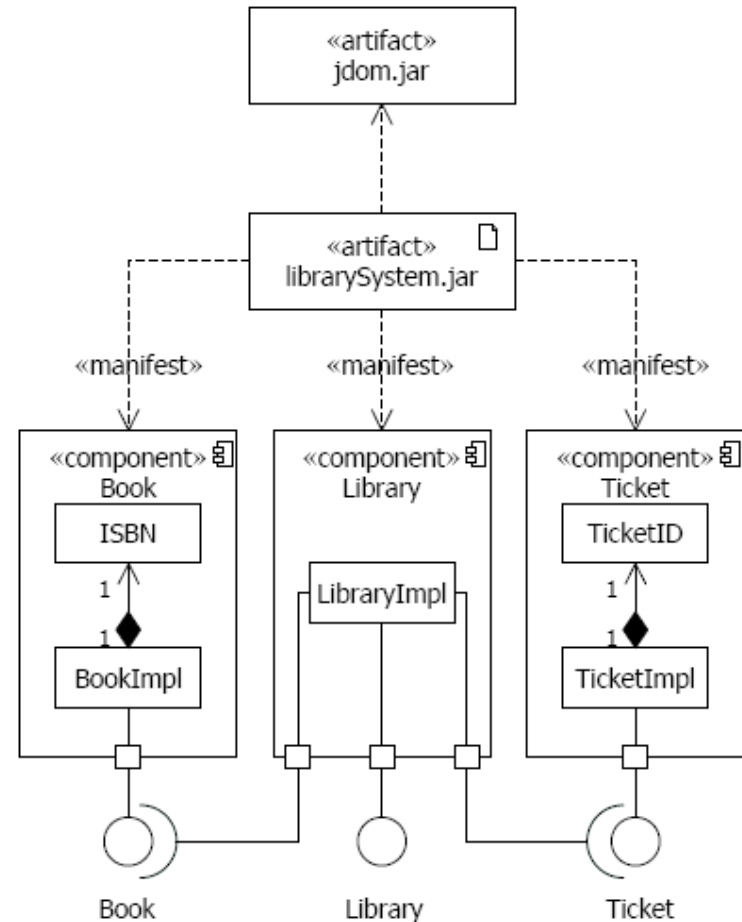
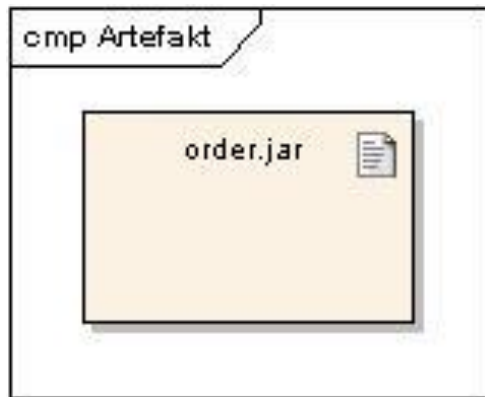


Vztahy mezi komponentami

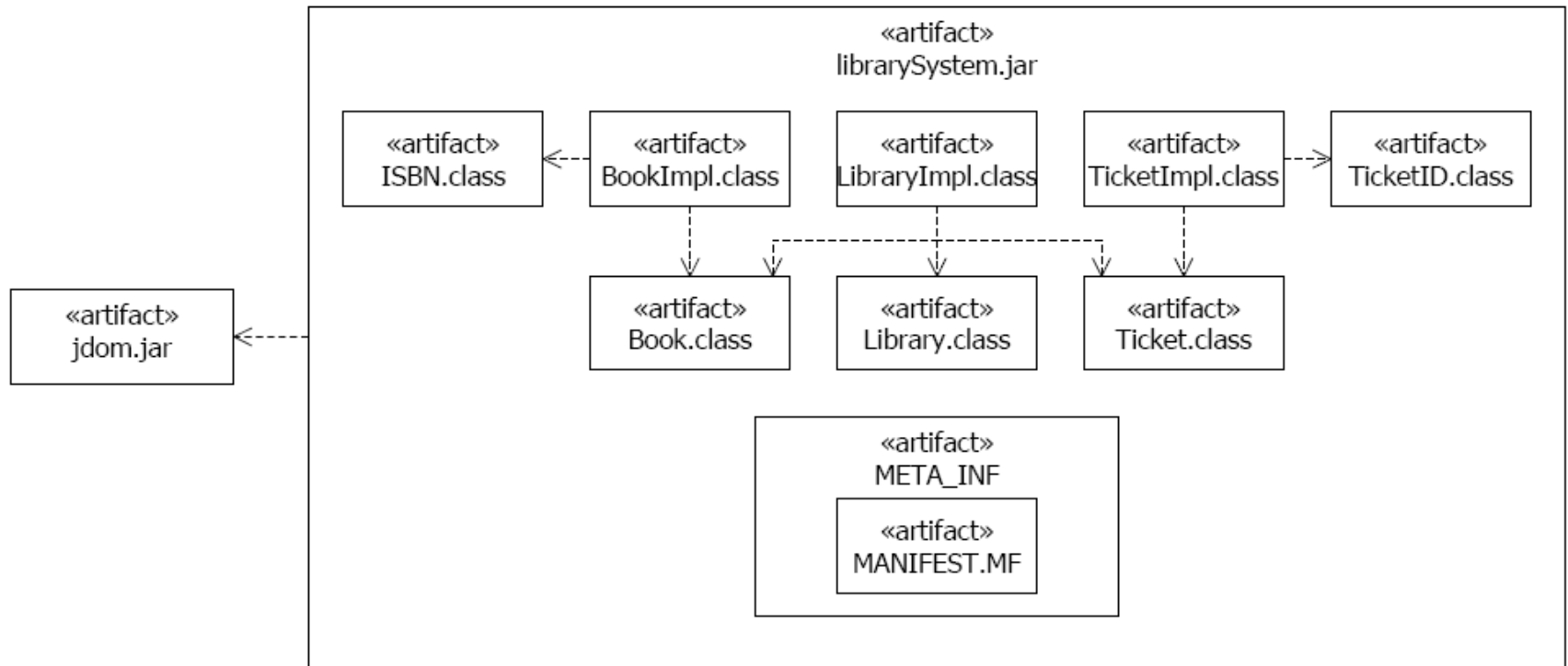


Artefakt

Fyzický projev jedné či více komponent - zdrojové soubory, spustitelné soubory, skripty, dokumenty, ...



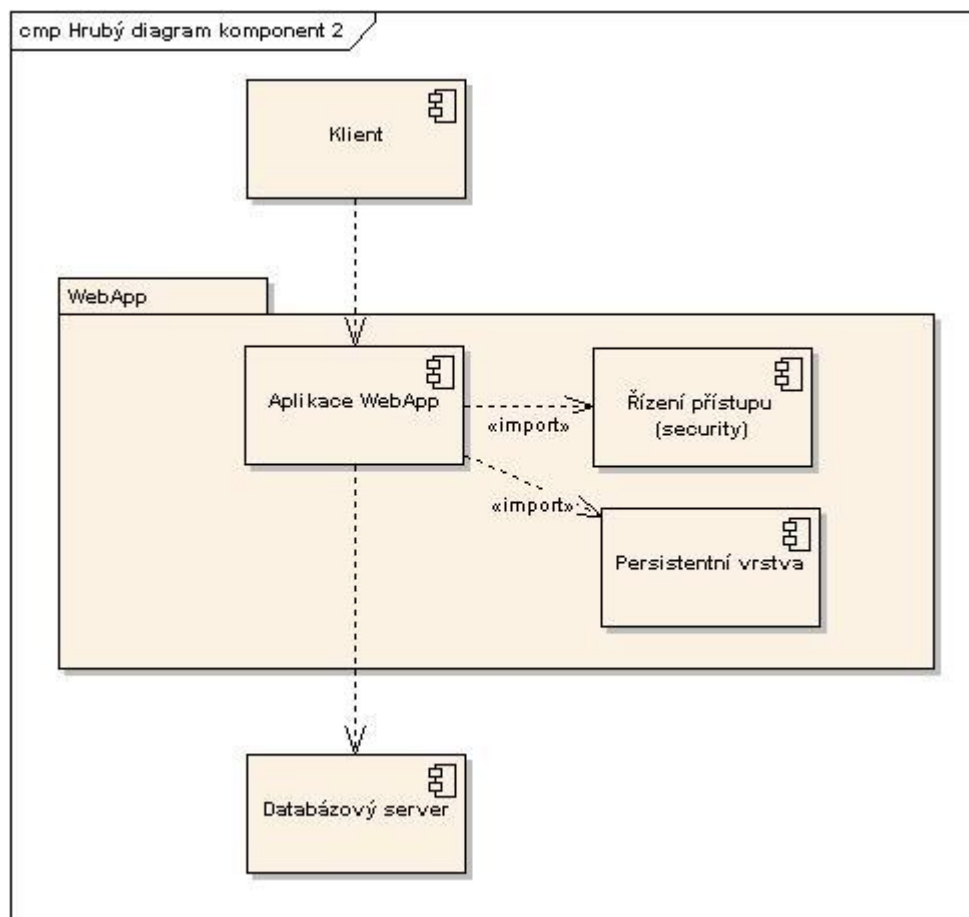
Vztahy mezi artefakty



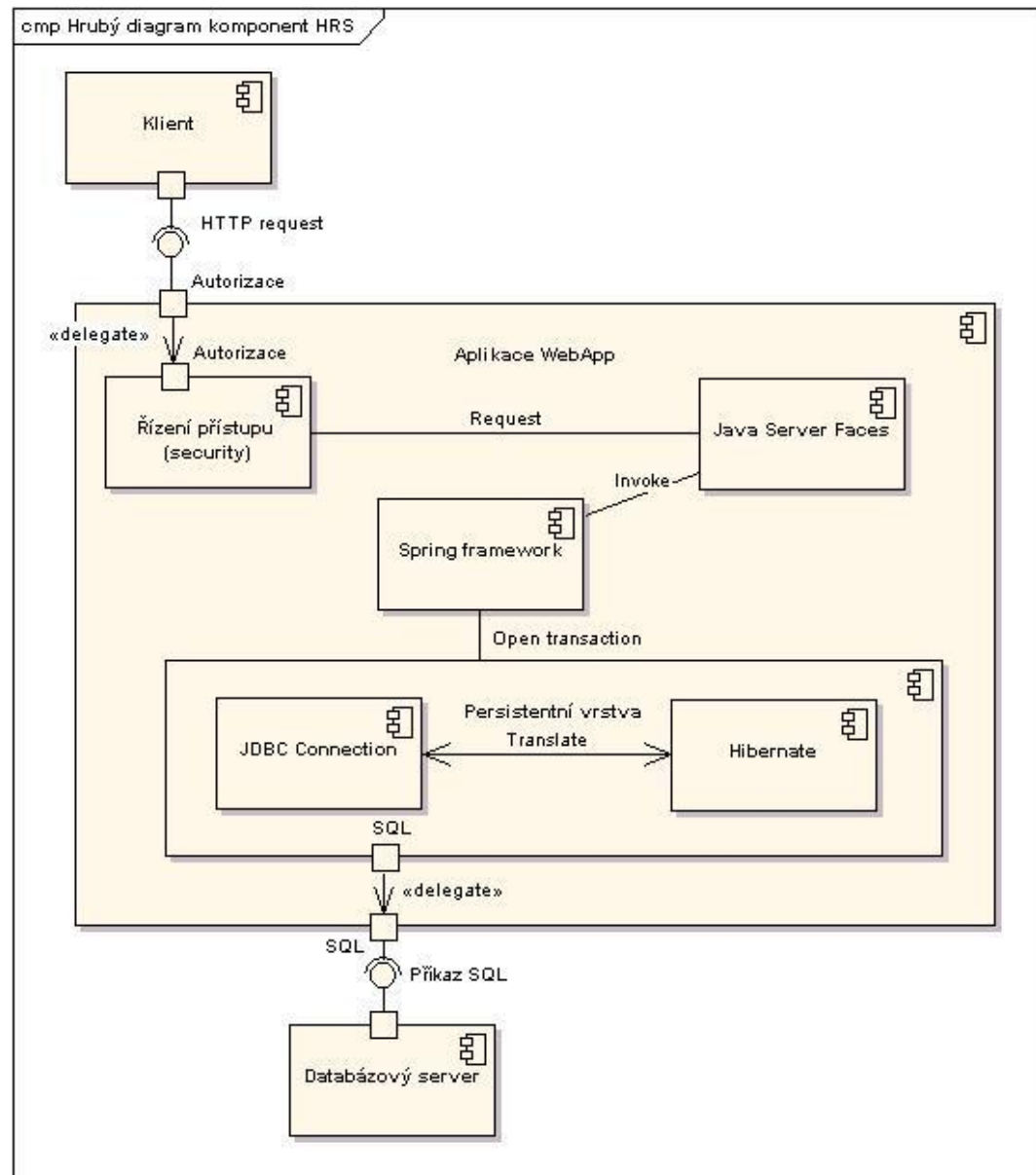
Standardní stereotypy artefaktů

artifact stereotype	semantics
«file»	A fyzický soubor
«deployment spec»	specifikace nasazení (e.g. web.xml in J2EE)
«document»	dokument
«executable»	spustitelný soubor
«library»	statická nebo dynamicky link. knihovna (DLL) nebo Java Archive (JAR)
«script»	skript proveditelný pomocí interpretu
«source»	Zkompilovatelný zdrojový soubor

Př. Hrubý diagram komponent HRS



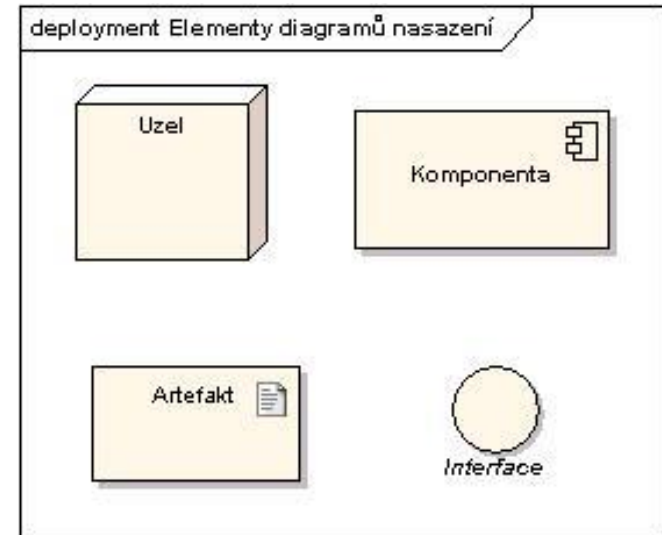
Př. O něco jemnější diagram komponent HRS



Diagramy nasazení (deployment)

- Co všichni známe:

Ukazují přiřazení artefaktů uzlům resp. ukazuje přiřazení instancí artefaktů instancím uzlů (diagram konkrétního nasazení).



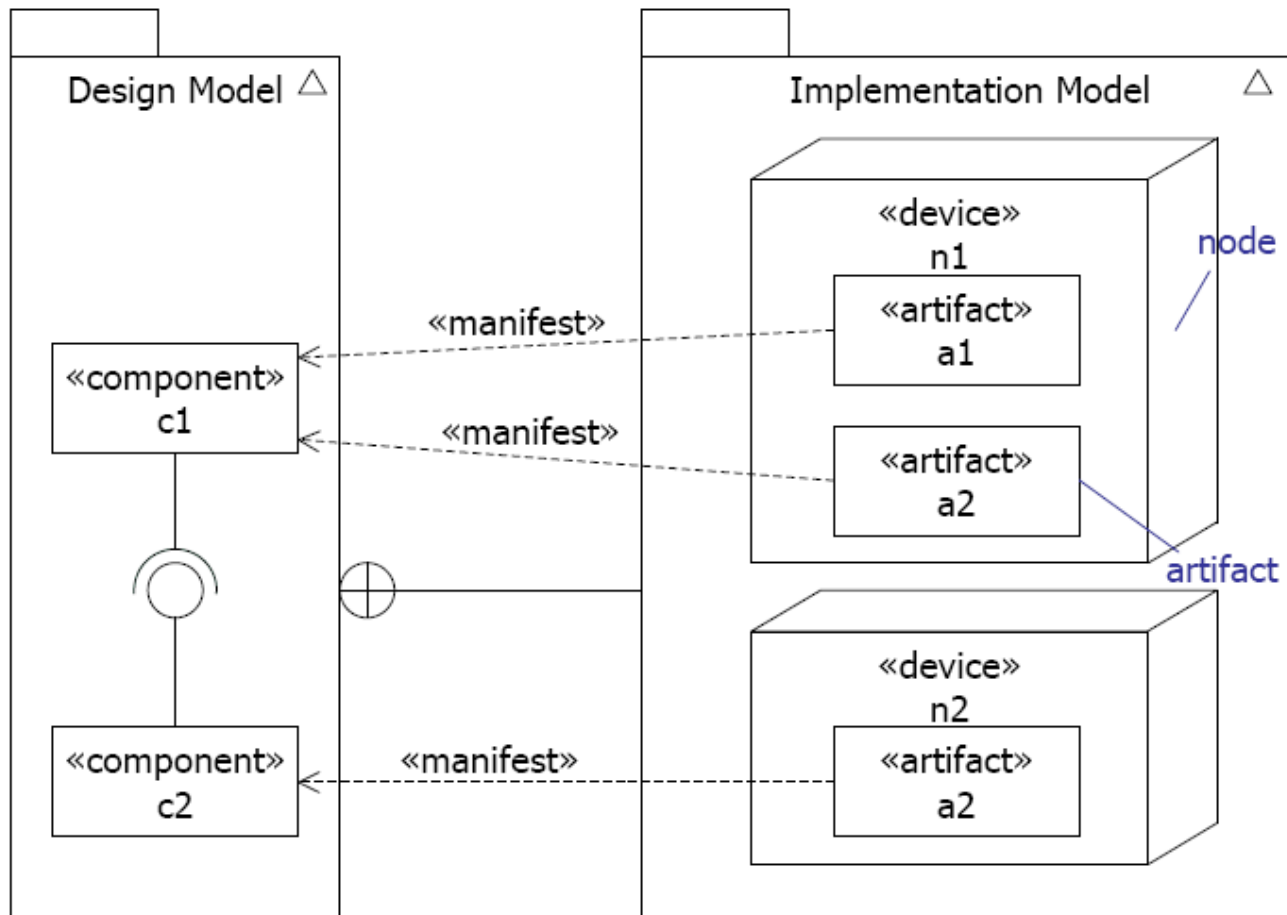
Uzel

- HW (server, PC, pokladna,...) - <<device>>
- SW (OS, www server, aplikační server,...) - <<environment>>

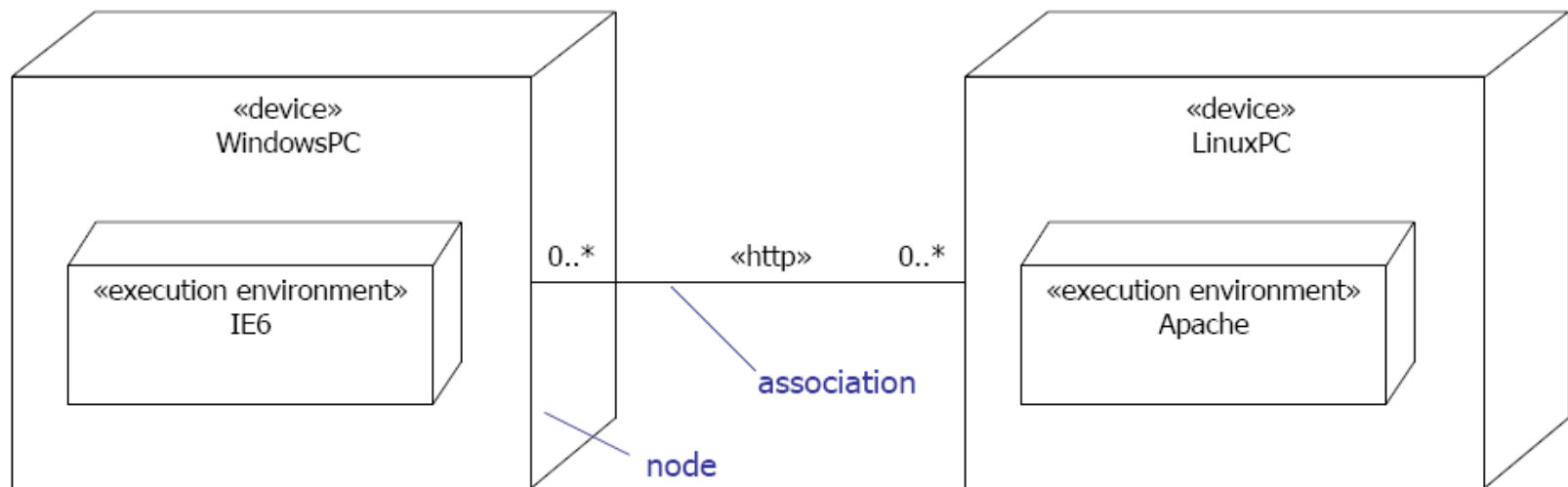
Artefakt

- obvykle projev jedné či více komponent, či fyzickým projevem libovolného prvku UML

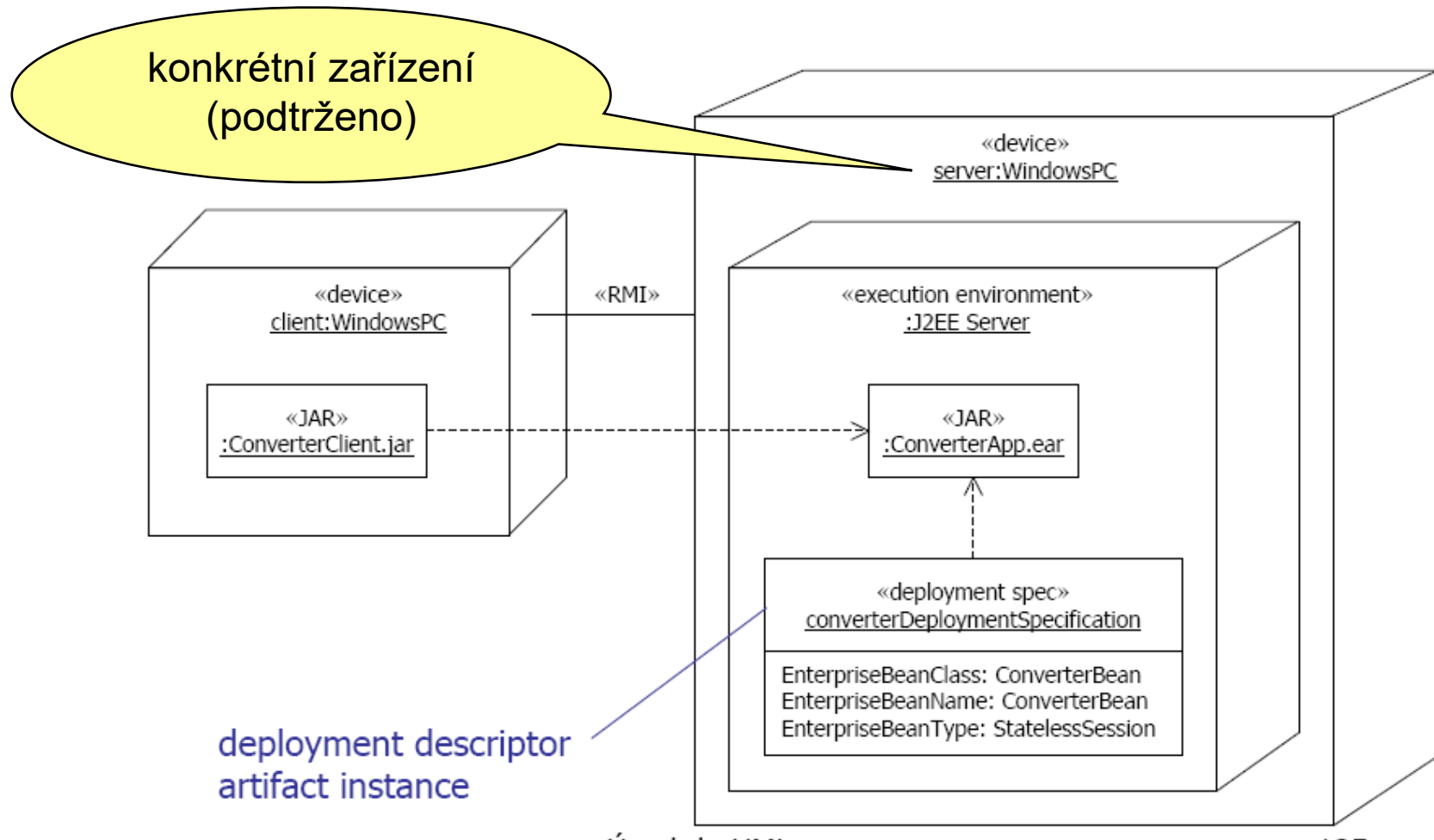
Nasazení versus komponenty



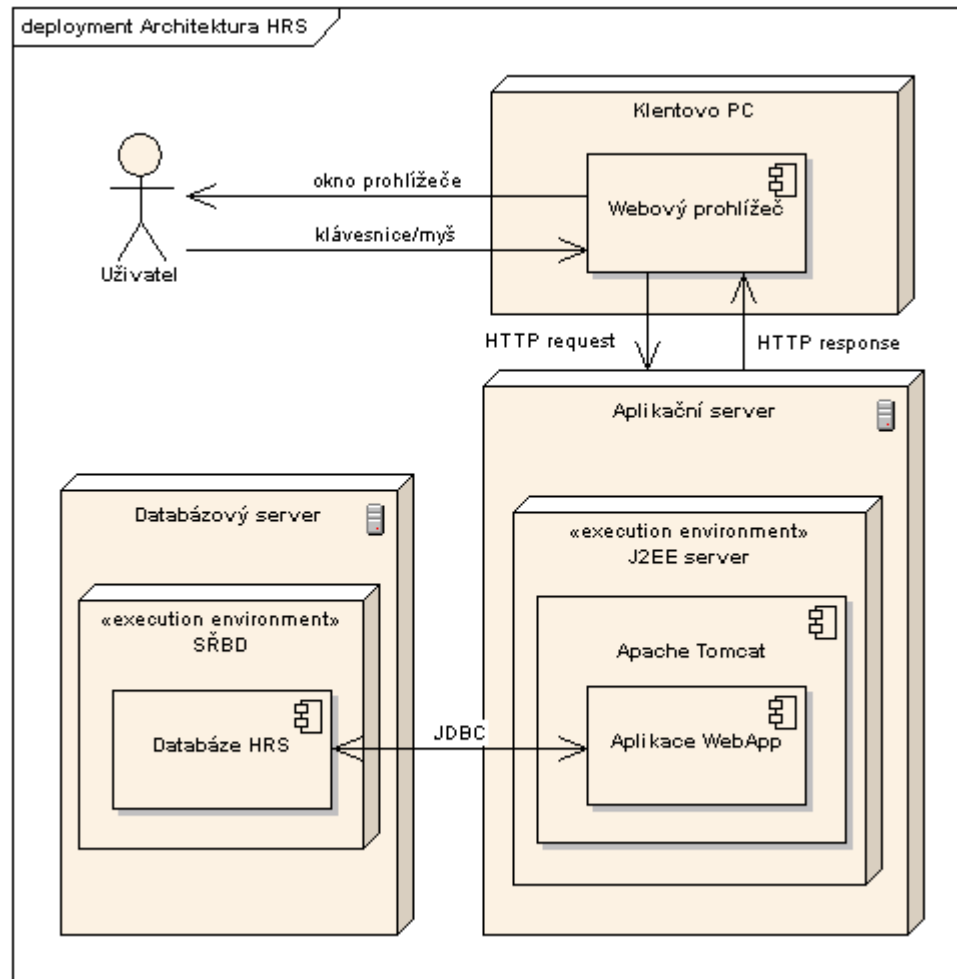
Typy uzlů



Nasazení artefaktů na uzly

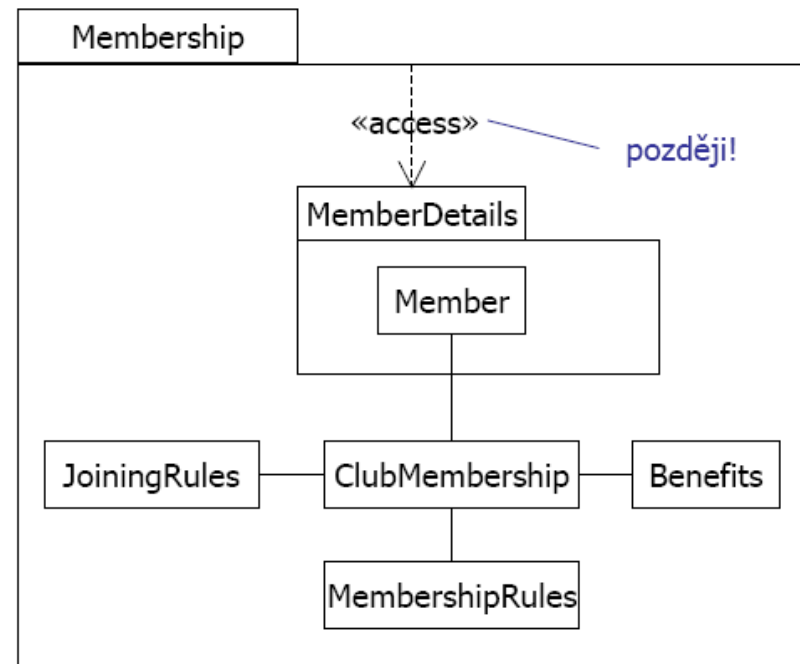
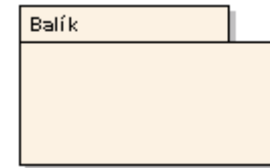
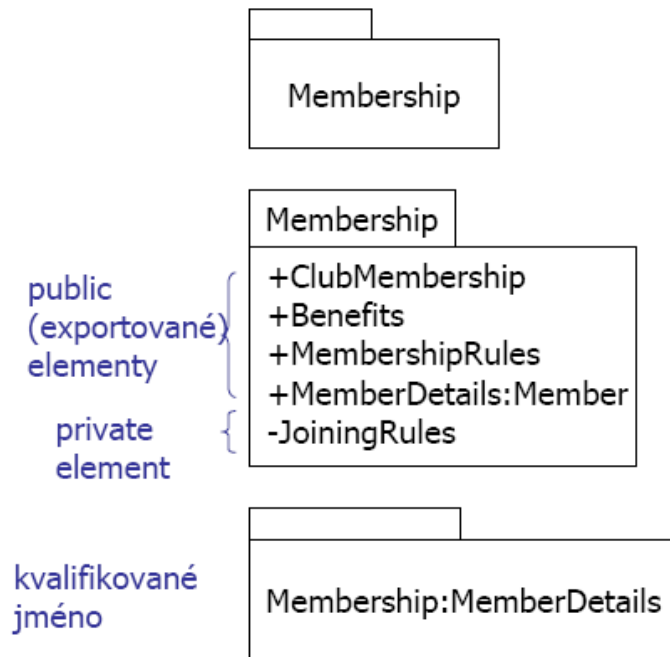


Příklad: Architektura HRS

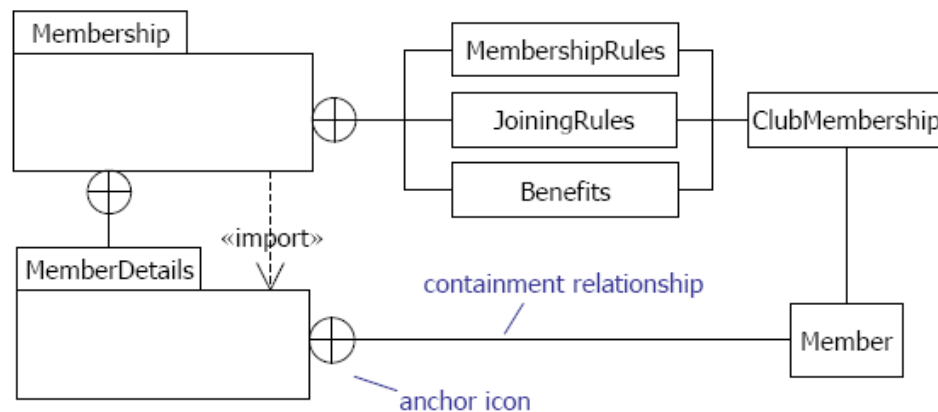
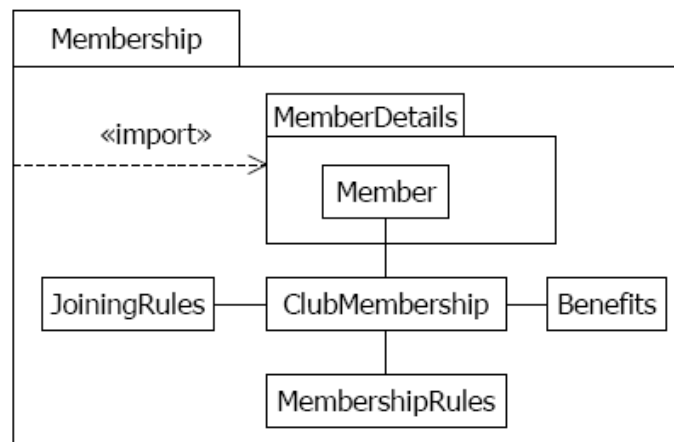


Diagramy balíků

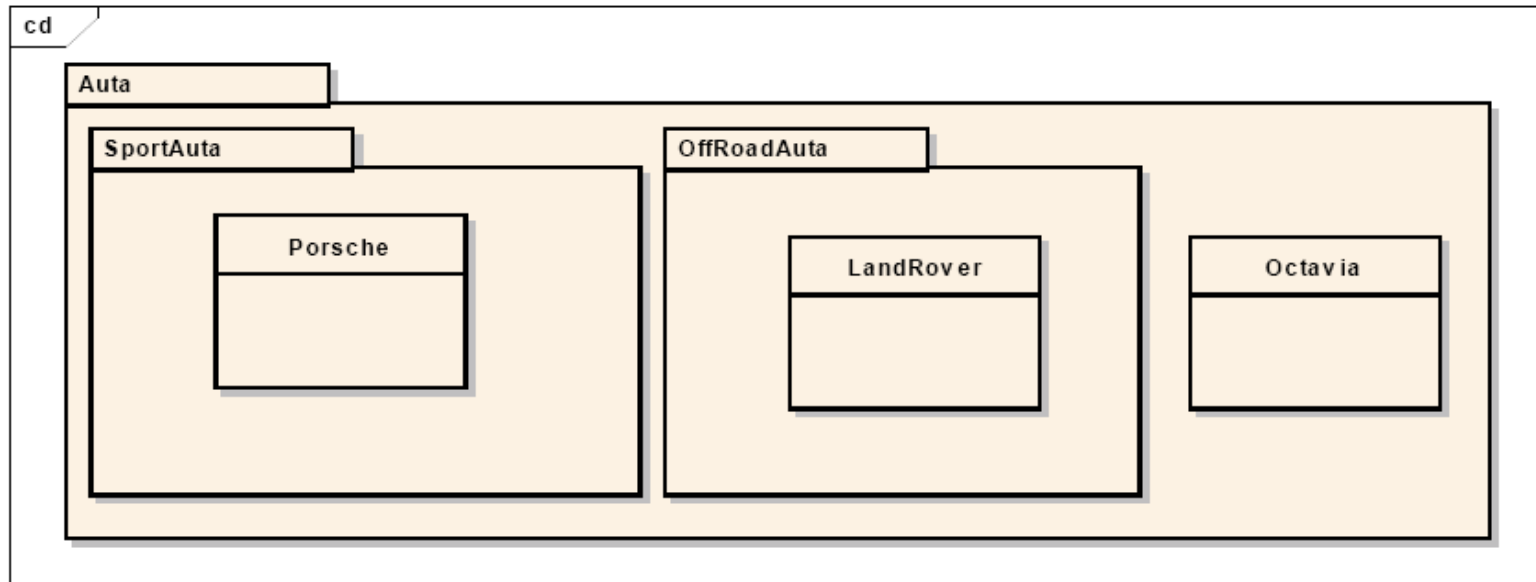
- Co všichni známe:



Vnořené balíčky

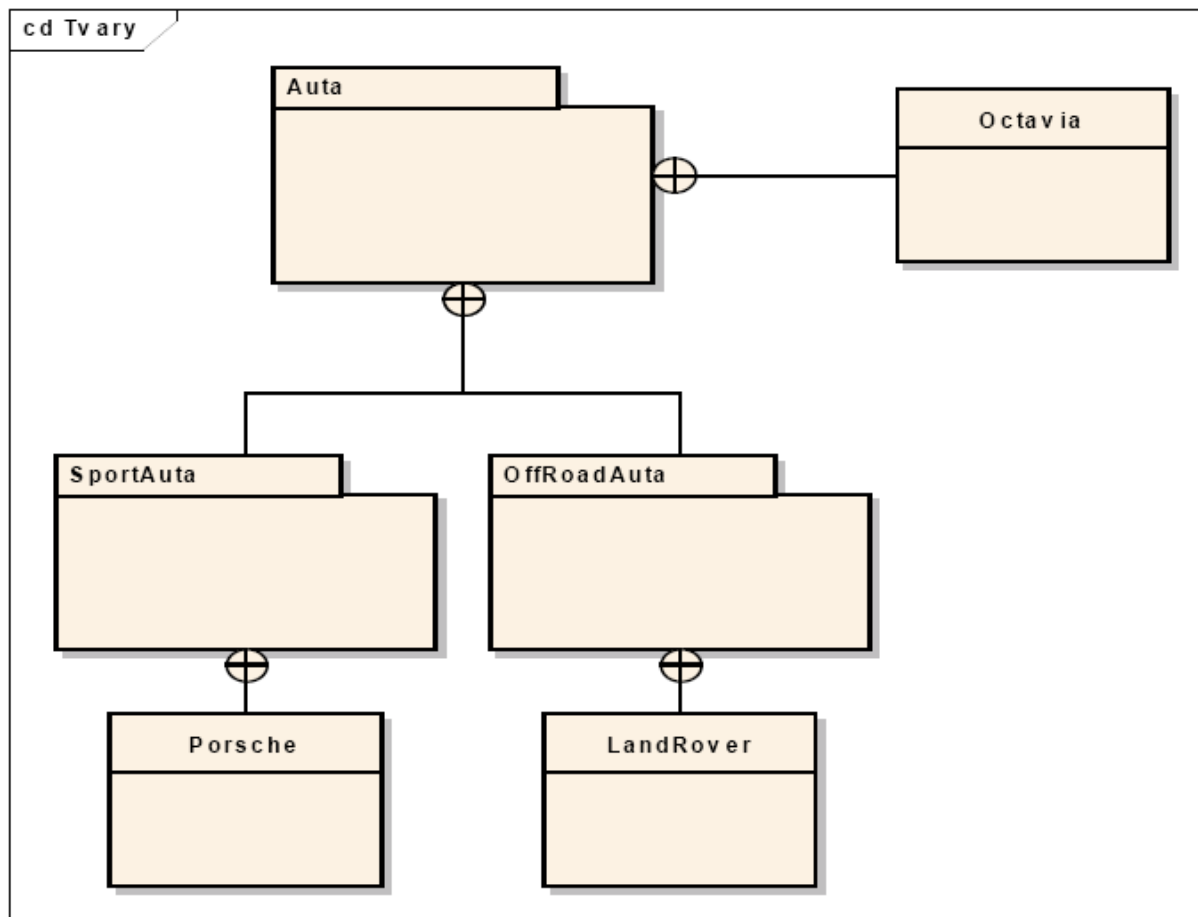


Vnoření a jmenné prostory



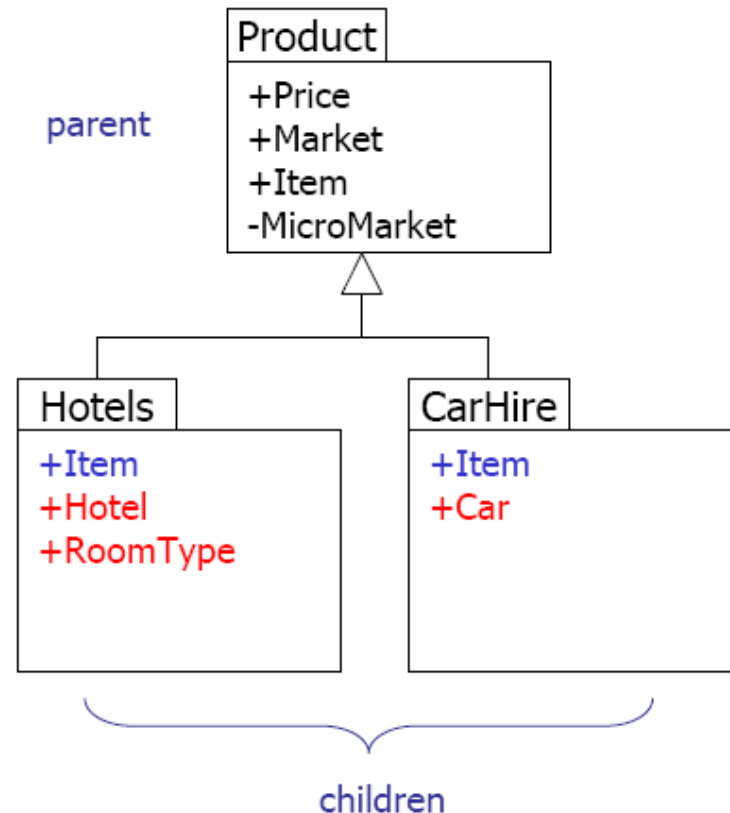
- Jmenné prostory
 - `Auta::SportAuta::Porsche`
 - `Auta::OffRoadAuta::LandRover`
 - `Auta::Octavia`
- v rámci balíčku – bez prefixu

Vnoření a jmenné prostory II.



Generalizace balíčků

- Potomci mohou přetížit prvky z předka.
- Potomek dědí veřejné (+) a chráněné (#) prvky.
- Potomci mohou přidat nové prvky.



The End